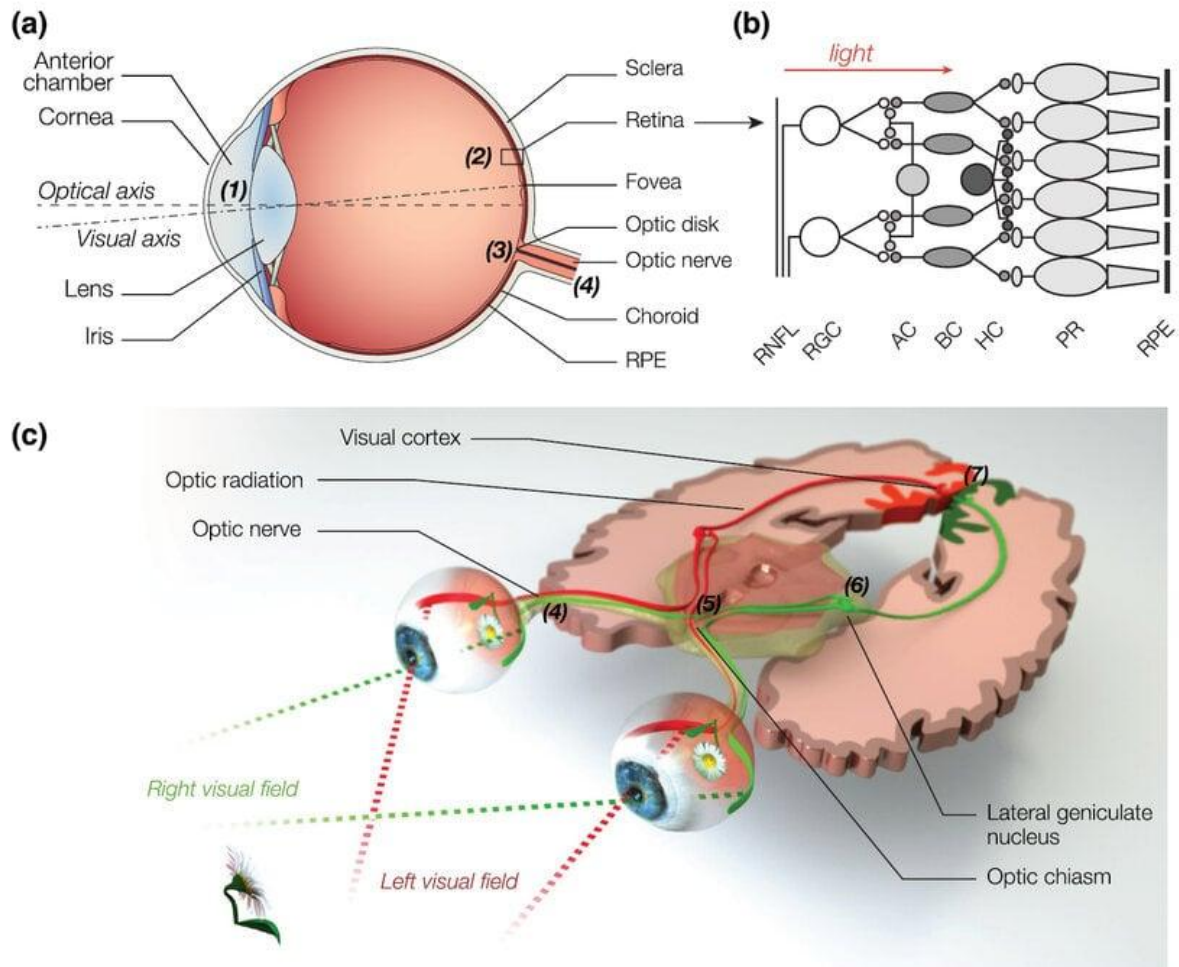


Convolutional Neural Network (CNN)



সূচিপত্র (Table of Contents)

Contents

সূচিপত্র (Table of Contents)	2
অধ্যায় ১ — CNN কী এবং কেন দরকার?	5
১.১ সাধারণ ANN দিয়ে ছবি প্রসেস করলে কী সমস্যা হয়?	5
১.২ CNN-এর সমাধান	5
অধ্যায় ২ — CNN ও মানুষের ভিজ্যুয়াল সিস্টেমের সাদৃশ্য	6
২.১ মানুষের ভিজ্যুয়াল পাথওয়ে	6
২.২ Hubel ও Wiesel-এর পরীক্ষা (১৯৬০-এর দশক)	6
(ক) Simple Cell	7
(খ) Complex Cell	7
২.৩ CNN-এর সাথে তুলনা	7
অধ্যায় ৩ — সংক্ষিপ্ত ইতিহাস	8
অধ্যায় ৪ — ছবির মৌলিক ধারণা (Image Fundamentals)	8
৪.১ Pixel এবং Grayscale ছবি	9
৪.২ RGB (রঙিন) ছবি	9
অধ্যায় ৫ — Local Connectivity, Receptive Field ও Parameter Sharing	10
৫.১ Local Connectivity	10
৫.২ Receptive Field	10
৫.৩ Parameter Sharing	10
অধ্যায় ৬ — Convolution Operation	11
৬.১ মূল প্রবাহ	11
৬.২ Filter / Kernel কী?	11
৬.৩ Edge Detection — Convolution-এর একটা সহজ উদাহরণ	11
Vertical Edge Detection Filter	12
Horizontal Edge Detection Filter	12
অধ্যায় ৭ — Convolution-এর ধাপে ধাপে সংখ্যাসহ হিসাব	12
৭.১ গাণিতিক সংজ্ঞা	12
৭.২ পূর্ণাঙ্গ সংখ্যাসহ উদাহরণ	13
অবস্থান (1,1) — X-এর উপরের-বাঁ 3×3 অংশ	13
অবস্থান (1,2) — এক ঘর ডানে সরে	13
অবস্থান (2,1) — এক সারি নিচে নেমে	14
অবস্থান (2,2)	14
চূড়ান্ত Feature Map	14
অধ্যায় ৮ — Feature Map, Bias এবং Activation Function	15

৮.১ Feature Map	15
৮.২ Bias	15
৮.৩ Activation Function — ReLU	15
অধ্যায় ৯ — Output Size-এর সূত্র	15
অধ্যায় ১০ — Padding	16
১০.১ কেন Padding দরকার?	16
সমস্যা ১ — Feature Map ক্রমাগত ছোট হতে থাকে	16
সমস্যা ২ — ছবির প্রান্তের (border) তথ্য কম গুরুত্ব পায়	16
১০.২ সমাধান — Zero Padding	16
১০.৩ Padding সহ Output Size-এর সূত্র	16
১০.৪ Padding-এর দুইটা প্রচলিত ধরন (Keras-এর পরিভাষায়)	16
অধ্যায় ১১ — Stride	18
১১.১ সংজ্ঞা	18
১১.২ সাধারণ সূত্র	18
১১.৩ Floor ([]) কেন দরকার?	18
১১.৪ Stride ব্যবহারের কারণ	18
অধ্যায় ১২ — RGB / বহু-চ্যানেল ইনপুটে Convolution	19
অধ্যায় ১৩ — একাধিক Filter (Multiple Filters)	19
অধ্যায় ১৪ — Pooling	20
১৪.১ Convolution-এর পরেও সমস্যা কী থেকে যায়?	20
সমস্যা ১ — Memory ও Computation	20
সমস্যা ২ — Translation Variance	20
১৪.২ সমাধান — Down Sampling	20
১৪.৩ Pooling-এর প্রকারভেদ	21
১৪.৪ Max Pooling — সংখ্যাসহ উদাহরণ	21
১৪.৫ Pooling-এর Output Size	22
১৪.৬ Pooling বহু-চ্যানেল (Volume) Feature Map-এর উপর	23
১৪.৭ Translation Invariance — একটি সতর্ক ও সঠিক ব্যাখ্যা	23
১৪.৮ Pooling-এর সুবিধা (Advantages)	23
১৪.৯ Pooling-এর অসুবিধা (Disadvantages)	23
১৪.১০ Global Pooling	24
অধ্যায় ১৫ — CNN Architecture	24
১৫.১ LeNet-5 — একটি সম্পূর্ণ ক্লাসিক উদাহরণ	25
১৫.২ বিখ্যাত CNN আর্কিটেকচারসমূহ (সংক্ষিপ্ত)	27
অধ্যায় ১৬ — CNN বনাম ANN	27

১৬.১ মিল (Similarities).....	27
১৬.২ পার্থক্য (Differences).....	28
অধ্যায় ১৭ — Trainable Parameters গণনা.....	28
অধ্যায় ১৮ — CNN-এর সম্পূর্ণ Forward Propagation.....	29
১৮.১ নোটেশন (Notation).....	29
১৮.২ Forward Propagation সমীকরণসমূহ.....	29
১৮.৩ সম্পূর্ণ সংখ্যাসহ Forward Propagation উদাহরণ.....	30
১৮.৪ Max Pooling, Flatten, এবং FC (সাধারণ ধারাবাহিকতা).....	31
অধ্যায় ১৯ — Loss Function ও Gradient Descent.....	32
১৯.১ Loss Function — Binary Cross Entropy.....	32
১৯.২ Gradient Descent.....	32
অধ্যায় ২০ — Backpropagation: ধারণা ও কৌশল.....	33
২০.১ মূল অন্তর্দৃষ্টি — Forward বনাম Backward.....	33
২০.২ কৌশল — নেটওয়ার্ককে দুই ভাগে ভাগ করা.....	33
অধ্যায় ২১ — Backpropagation পর্বক: ANN অংশ (W_2, b_2).....	34
২১.১ Chain Rule সেটআপ.....	34
অধ্যায় ২২ — Backpropagation পর্বক: CNN অংশ (W_1, b_1).....	37
অধ্যায় ২৩ — Convolution Layer-এর Gradient (সবচেয়ে গুরুত্বপূর্ণ অংশ).....	41
অধ্যায় ২৪ — সম্পূর্ণ Backpropagation প্রবাহ (সারসংক্ষেপ).....	45
অধ্যায় ২৫ — সম্পূর্ণ সংখ্যাসহ Backpropagation উদাহরণ.....	46

অধ্যায় ১ — CNN কী এবং কেন দরকার?

Convolutional Neural Network (CNN) হলো একটি বিশেষ ধরনের Neural Network, যা মূলত ছবি (image), ভিডিও বা যেকোনো grid-আকৃতির (spatial) ডেটা প্রসেস করার জন্য ডিজাইন করা হয়েছে। CNN আসলে সাধারণ Artificial Neural Network (ANN)-এরই একটি বিশেষায়িত রূপ — একই ভিত্তি (neuron, weight, bias, activation, backpropagation) থাকলেও এতে দুটি নতুন ধরনের layer যুক্ত হয়: Convolution Layer এবং Pooling Layer।

১.১ সাধারণ ANN দিয়ে ছবি প্রসেস করলে কী সমস্যা হয়?

একটা সাধারণ ANN-এ input হিসেবে একটা 1D vector দিতে হয়। একটা রঙিন ছবি, ধরা যাক আকার $228 \times 228 \times 3$ (উচ্চতা $\times$ প্রস্থ $\times$ channel), সরাসরি ANN-এ দিতে গেলে প্রথমে সেটাকে flatten করে একটা লম্বা vector-এ রূপান্তর করতে হবে:

$$228 \times 228 \times 3 = 1,55,952 \text{ টি সংখ্যার vector}$$

এই বিশাল vector-কে যদি প্রথম hidden layer-এর মাত্র ১০০০টা neuron-এর সাথে fully connect করা হয়, তাহলে প্রথম layer-এই প্যারামিটার সংখ্যা দাঁড়ায়:

$$1,55,952 \times 1000 \approx ১৫ \text{ কোটি } ৬০ \text{ লক্ষ ওজন (weight)}$$

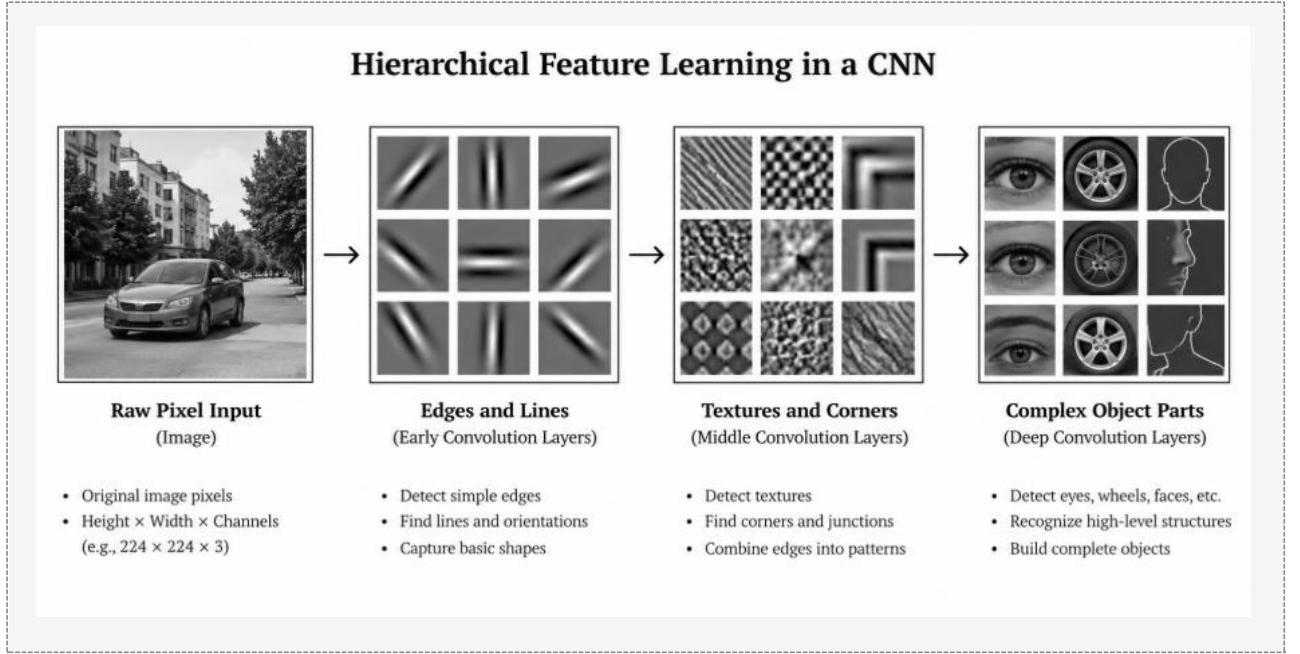
এত বিশাল সংখ্যক প্যারামিটার প্রশিক্ষণ (training) করতে বিপুল পরিমাণ ডেটা ও কম্পিউটেশন দরকার হয়, এবং overfitting-এর ঝুঁকি বহুগুণ বেড়ে যায়। এর পাশাপাশি আরেকটি গুরুত্বপূর্ণ সমস্যা হলো — flatten করার সময় ছবির spatial structure (অর্থাৎ কোন pixel কোন pixel-এর পাশে আছে, উপরে আছে, নিচে আছে — এই স্থানিক সম্পর্ক) সম্পূর্ণ নষ্ট হয়ে যায়। অথচ একটা ছবিতে বস্তু চেনার জন্য pixel-গুলোর মধ্যকার এই প্রতিবেশী সম্পর্কটাই সবচেয়ে গুরুত্বপূর্ণ তথ্য।

১.২ CNN-এর সমাধান

CNN এই দুটি সমস্যা সমাধান করে দুটি মূল ধারণার মাধ্যমে — যা পরবর্তী অধ্যায়গুলোতে বিস্তারিত আলোচনা করা হবে:

ধারণা	সংক্ষিপ্ত ব্যাখ্যা
Local Connectivity	প্রতিটা neuron পুরো ছবি না দেখে ছবির একটা ছোট অংশ (local region) দেখে
Parameter Sharing	একই filter (একই সেট weight) পুরো ছবির উপর বারবার ব্যবহার হয়, ফলে প্যারামিটার সংখ্যা অনেক কমে যায়

এর ফলে CNN ছবির spatial structure অক্ষত রেখেই অনেক কম প্যারামিটার দিয়ে শক্তিশালী feature শিখতে পারে।



অধ্যায় ২ — CNN ও মানুষের ভিজুয়াল সিস্টেমের সাদৃশ্য

CNN-এর মূল স্থাপত্যিক ধারণাটি মানুষ ও অন্যান্য স্তন্যপায়ী প্রাণীর মস্তিষ্কের ভিজুয়াল প্রসেসিং সিস্টেম থেকে অনুপ্রাণিত। এটি বোঝা গুরুত্বপূর্ণ, কারণ এখান থেকেই "hierarchical" এবং "local receptive field" — এই দুটি ধারণার উৎপত্তি।

২.১ মানুষের ভিজুয়াল পাথওয়ে

চোখে আলো পড়ার পর মস্তিষ্কে তথ্য প্রক্রিয়াকরণ কয়েকটি ধাপে ঘটে:

ধাপ	কাজ
Retina (রেটিনা)	চোখের পর্দা — আলোকে electrical signal-এ রূপান্তর করে
Optic Nerve (অপটিক নার্ভ)	সেই সিগন্যাল মস্তিষ্কের দিকে বহন করে
Thalamus / LGN	সিগন্যালের প্রাথমিক preprocessing করে, cortex-এ পাঠানোর আগে
Visual Cortex (V1)	প্রথম মূল ভিজুয়াল প্রসেসিং অঞ্চল, যেখান থেকে হায়ারার্কিক্যাল বিশ্লেষণ শুরু হয়

এখানে লক্ষণীয় বিষয় হলো — মস্তিষ্ক একবারেই পুরো দৃশ্য "বুঝে ফেলে না"; বরং স্তরে স্তরে (hierarchically), ধাপে ধাপে প্রক্রিয়াকরণ করে সরল থেকে জটিল তথ্যে পৌঁছায়। CNN-ও ঠিক এই কৌশলটিই অনুসরণ করে — একের পর এক Convolution layer দিয়ে সরল feature থেকে জটিল feature-এ পৌঁছানো হয়।

২.২ Hubel ও Wiesel-এর পরীক্ষা (১৯৬০-এর দশক)

David Hubel এবং Torsten Wiesel বিড়ালের (cat) ভিজ্যুয়াল কর্টেক্সে ইলেকট্রোড বসিয়ে পরীক্ষা করেন — বিভিন্ন আকৃতি ও কোণের আলোকরেখা দেখালে কোন নিউরন কখন সক্রিয় (fire) হয়। এই গবেষণার জন্য তাঁরা ১৯৮১ সালে নোবেল পুরস্কার পান। তাঁদের পরীক্ষায় ভিজ্যুয়াল কর্টেক্সে দুই ধরনের কোষ (cell) পাওয়া যায়:

(ক) Simple Cell

Simple Cell একটি নির্দিষ্ট orientation (কোণ)-এর edge-এর প্রতি সংবেদনশীল — যেমন কোনো একটা কোষ শুধু উল্লম্ব (vertical) রেখা দেখলেই সাড়া দেয়, অন্য কোণের রেখায় দেয় না। এটির receptive field (যে এলাকাটুকু একটা কোষ "দেখতে" পারে) ছোট, এবং এটি মূলত একটি basic-level feature detector — অনেকটা edge detector-এর মতো।

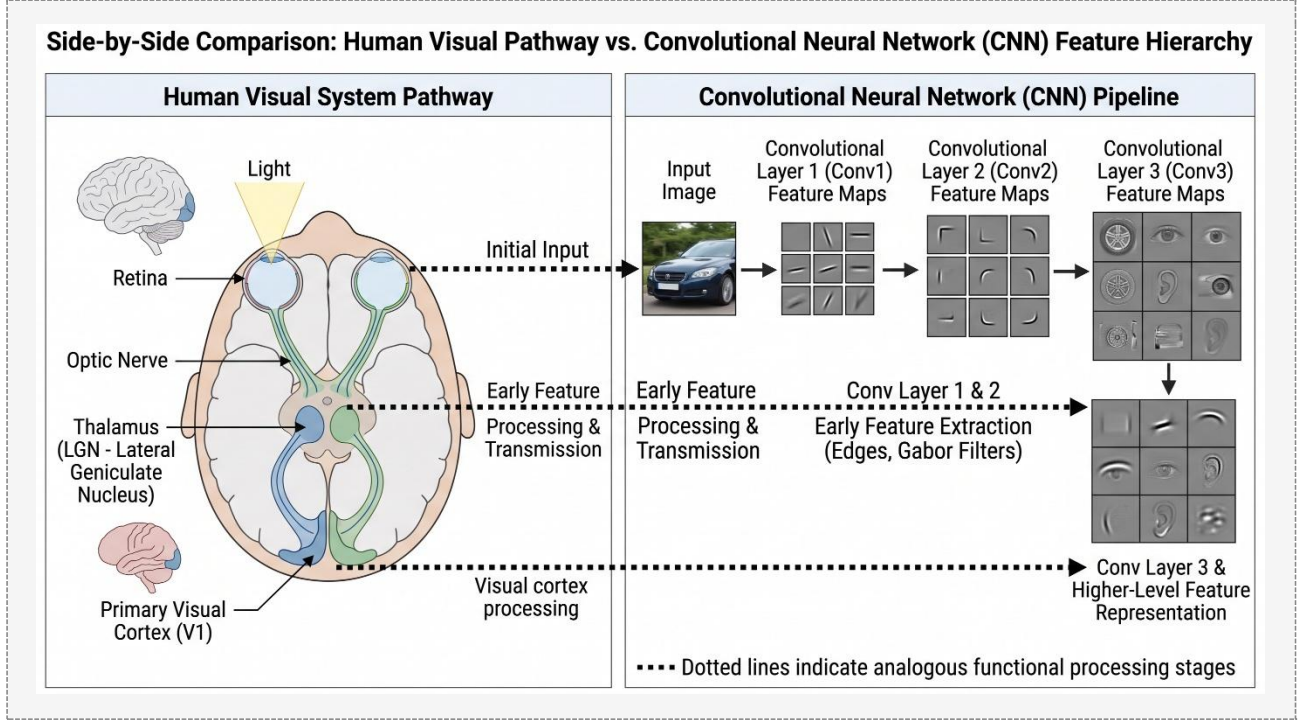
(খ) Complex Cell

Complex Cell একাধিক Simple Cell-এর output একত্র করে আরও জটিল প্যাটার্ন শনাক্ত করে। এর receptive field তুলনামূলক বড়, এবং গুরুত্বপূর্ণভাবে — বস্তুটির অবস্থান সামান্য সরে গেলেও (shift হলেও) Complex Cell একইভাবে সাড়া দিতে পারে। এই বৈশিষ্ট্যটিই CNN-এর translation invariance ধারণার জৈবিক ভিত্তি।

২.৩ CNN-এর সাথে তুলনা

Visual Cortex	CNN
Simple Cell (edge-sensitive)	প্রথম দিকের Convolution Layer
Complex Cell (pattern-combining)	গভীরতর Convolution Layer
Receptive Field	Filter / Kernel-এর কভারেজ এলাকা
Preferred stimulation প্যাটার্ন	Training-এর মাধ্যমে শেখা filter-এর weight প্যাটার্ন
Hierarchical (স্তরভিত্তিক) প্রসেসিং	একের পর এক Convolution Layer

মনে রাখো: এই তুলনাটি একটি অনুপ্রেরণামূলক (inspirational) analogy — CNN আক্ষরিক অর্থে মস্তিষ্কের কপি নয়। CNN হলো একটি গাণিতিক মডেল যা এই জৈবিক পর্যবেক্ষণ থেকে স্থাপত্যিক (architectural) ধারণা ধার করেছে।



অধ্যায় ৩ — সংক্ষিপ্ত ইতিহাস

CNN-এর ইতিহাস সংক্ষেপে বোঝা দরকার, কারণ এটি বোঝা যায় কেন আজকের CNN এভাবে ডিজাইন করা হয়েছে।

সময়কাল	ঘটনা
১৯৮৯-১৯৯৮	Yann LeCun ও তাঁর সহকর্মীরা Backpropagation-কে Convolution architecture-এর সাথে যুক্ত করেন এবং LeNet তৈরি করেন, যা হাতে লেখা সংখ্যা (digit) চিনতে ব্যবহৃত হতো (যেমন ব্যাংক চেকের সংখ্যা পড়া)।
২০১২	AlexNet নামের একটি গভীর CNN ImageNet প্রতিযোগিতায় প্রতিদ্বন্দ্বীদের তুলনায় বিশাল ব্যবধানে জয়লাভ করে। এই ঘটনাটি Deep Learning বিপ্লবের অন্যতম সূচনাবিন্দু হিসেবে বিবেচিত।
২০১২-পরবর্তী	VGG, GoogLeNet (Inception), ResNet-সহ একের পর এক আরও গভীর ও কার্যকর CNN আর্কিটেকচার আবিষ্কৃত হয়, এবং CNN computer vision-এর প্রধান হাতিয়ারে পরিণত হয়।

মূল অন্তর্দৃষ্টি: LeNet-এর সবচেয়ে গুরুত্বপূর্ণ অবদান হলো এই প্রমাণ যে, convolution filter-এর ভেতরের সংখ্যাগুলো হাতে ডিজাইন না করে Backpropagation দিয়ে ডেটা থেকে স্বয়ংক্রিয়ভাবে শেখা সম্ভব — ঠিক যেমন ANN-এর weight শেখা হয়।

অধ্যায় ৪ — ছবির মৌলিক ধারণা (Image Fundamentals)

CNN বোঝার আগে বুঝে নেওয়া দরকার — কম্পিউটারের কাছে একটা ছবি আসলে কী। কম্পিউটারের কাছে একটা ছবি হলো নিছক একটা সংখ্যার ম্যাট্রিক্স (2D বা 3D array), যেখানে প্রতিটা সংখ্যা একটা pixel-এর উজ্জ্বলতা বা রঙ নির্দেশ করে।

8.1 Pixel এবং Grayscale ছবি

একটা grayscale (সাদা-কালো / ধূসর) ছবিতে প্রতিটা pixel-এর মান সাধারণত 0 থেকে 255-এর মধ্যে থাকে:

মান	অর্থ
0	সম্পূর্ণ কালো
255	সম্পূর্ণ সাদা
মাঝামাঝি মান	ধূসরের বিভিন্ন শেড

একটা grayscale ছবিতে channel সংখ্যা মাত্র ১টি। উদাহরণস্বরূপ, বিখ্যাত MNIST ডেটাসেটের প্রতিটা digit ছবির আকার:

$$28 \times 28 \times 1 = 784 \text{ টি সংখ্যা}$$

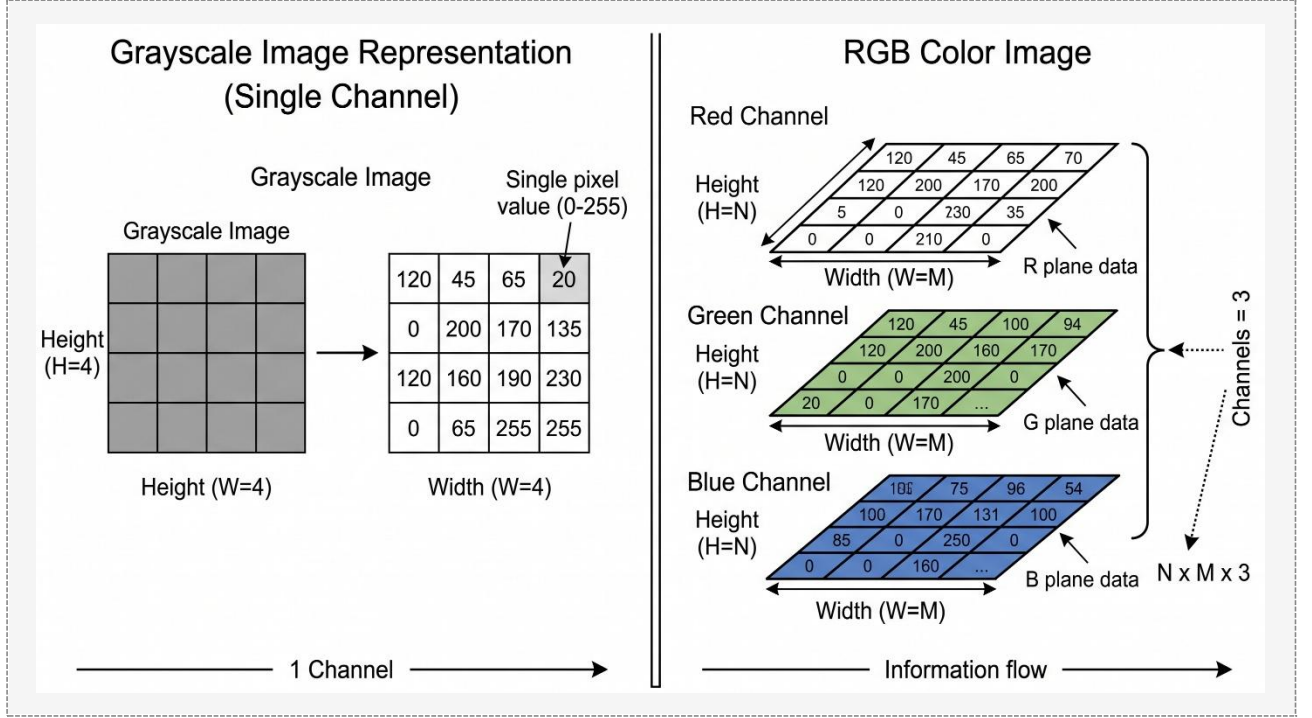
8.2 RGB (রঙিন) ছবি

একটা রঙিন ছবিতে তিনটি channel থাকে — Red (R), Green (G), এবং Blue (B)। প্রতিটা channel আসলে একটা আলাদা 2D ম্যাট্রিক্স; তিনটি ম্যাট্রিক্স একটার উপর আরেকটা স্তরের (layer/depth) মতো সাজানো থাকে। তাই একটা RGB ছবির আকার লেখা হয়:

$$\text{উচ্চতা (Height)} \times \text{প্রস্থ (Width)} \times \text{চ্যানেল (Channel)}$$

উদাহরণস্বরূপ, একটা 228×228 আকারের রঙিন ছবির সম্পূর্ণ আকার হবে $228 \times 228 \times 3$, অর্থাৎ মোট সংখ্যা:

$$228 \times 228 \times 3 = 1,55,952$$



অধ্যায় ৫ — Local Connectivity, Receptive Field ও Parameter Sharing

এই তিনটি ধারণা CNN-এর ভিত্তি — Convolution operation পুরোপুরি বোঝার আগে এই intuition-গুলো স্পষ্ট হওয়া দরকার।

৫.১ Local Connectivity

সাধারণ ANN-এ (Fully Connected Layer) একটা neuron ইনপুটের প্রতিটা মানের সাথে যুক্ত থাকে। কিন্তু CNN-এ একটা neuron (বা filter-এর একটা অবস্থান) ইনপুট ছবির শুধুমাত্র একটা ছোট, স্থানীয় (local) অংশের সাথে যুক্ত থাকে — পুরো ছবির সাথে নয়। একেই বলা হয় Local Connectivity বা Sparse Connectivity।

৫.২ Receptive Field

Receptive Field বলতে input ছবির সেই ছোট অংশকে বোঝায় যেটা একটি neuron/filter এক মুহূর্তে "দেখতে" পারে বা যেটার উপর ভিত্তি করে সে তার output গণনা করে। যেমন একটা 3x3 filter-এর receptive field হলো ছবির 3x3 এলাকা। নেটওয়ার্কের গভীরে গেলে (একাধিক Convolution layer-এর পর) কার্যকরী receptive field ক্রমান্বয়ে বড় হতে থাকে, কারণ পরের layer-এর একটা pixel আগের layer-এর একাধিক pixel-এর সমন্বয়ে তৈরি।

৫.৩ Parameter Sharing

Parameter Sharing হলো CNN-এর সবচেয়ে গুরুত্বপূর্ণ ও শক্তিশালী ধারণাগুলোর একটি: একই filter (একই সেট weight ও bias) পুরো ছবির বিভিন্ন অবস্থানে বারবার পুনঃব্যবহার (reuse) করা হয়। এর সুবিধা দুটি:

সুবিধা	ব্যাখ্যা
প্যারামিটার সংখ্যা কমে যায়	প্রতিটা অবস্থানের জন্য আলাদা weight না রেখে একটাই filter পুরো ছবিতে চলে, তাই মোট trainable parameter অনেক কম
Feature detection সর্বত্র সমানভাবে কাজ করে	একটা filter যদি "vertical edge" চিনতে শেখে, সে ছবির যেকোনো জায়গায় vertical edge চিনতে পারবে — অবস্থান-নিরপেক্ষ (position-independent) feature detection

মনে রাখো: ANN-এর একটা neuron-এর weighted sum গণনার সাথে তুলনা করলে: ANN-এ একটা neuron ইনপুটের সাথে একটাই weighted sum তৈরি করে। CNN-এ একটা filter ছবির প্রতিটা অবস্থানে একই ধরনের local weighted sum তৈরি করে — পার্থক্য শুধু এই যে CNN একই filter-এর weight-গুলো ছবির বিভিন্ন অঞ্চলে পুনরায় ব্যবহার (reuse) করে।

অধ্যায় ৬ — Convolution Operation

Convolution হলো CNN-এর কেন্দ্রীয় গাণিতিক অপারেশন। এটি একটি ছোট ম্যাট্রিক্স (Filter বা Kernel) কে বড় ইনপুট ছবির উপর দিয়ে স্লাইড করিয়ে, প্রতিটা অবস্থানে element-wise গুণ করে যোগফল বের করে একটি নতুন ম্যাট্রিক্স (Feature Map) তৈরি করে।

৬.১ মূল প্রবাহ

Input Image → *Filter* → *Element-wise Multiplication* → *Summation* → *Bias যোগ* → *Feature Map*

৬.২ Filter / Kernel কী?

Filter (একে Kernel-ও বলা হয়) একটা ছোট আকারের ম্যাট্রিক্স (সাধারণত 3×3 , 5×5 , বা 7×7), যার ভেতরের সংখ্যাগুলো (weight) নির্দিষ্ট একটা প্যাটার্ন (যেমন edge, texture) শনাক্ত করার জন্য দায়ী। গুরুত্বপূর্ণ কথা হলো — এই সংখ্যাগুলো আমরা হাতে বসাই না; এগুলো ANN-এর weight-এর মতোই trainable parameter এবং Backpropagation-এর মাধ্যমে training-এর সময় স্বয়ংক্রিয়ভাবে শেখা হয়। নেটওয়ার্ক নিজেই ঠিক করে কোন filter সবচেয়ে দরকারি feature ধরতে পারবে।

৬.৩ Edge Detection — Convolution-এর একটা সহজ উদাহরণ

Edge Detection বোঝার মাধ্যমে Convolution-এর কাজ সবচেয়ে সহজে বোঝা যায়। ছবিতে edge মানে হলো সেই জায়গা, যেখানে pixel-এর intensity (উজ্জ্বলতা) হঠাৎ পরিবর্তিত হয় — যেমন কালো থেকে সাদায় লাফ দেওয়া অংশ। Edge Detection মূলত দুই ধরনের:

Vertical Edge Detection Filter

একটা প্রচলিত vertical edge detector filter (3×3):

1	0	-1
1	0	-1
1	0	-1

এই filter-এর বাঁ কলাম ধনাত্মক এবং ডান কলাম ঋণাত্মক — অর্থাৎ এটি বাঁ ও ডান পাশের intensity-এর পার্থক্য মাপে, ফলে এটি উল্লম্ব (vertical) দিকের edge শনাক্ত করে।

Horizontal Edge Detection Filter

একটা প্রচলিত horizontal edge detector filter (3×3):

-1	-1	-1
0	0	0
1	1	1

এই filter-এর উপরের সারি ঋণাত্মক এবং নিচের সারি ধনাত্মক — এটি উপরে-নিচের intensity পার্থক্য মাপে, ফলে এটি আনুভূমিক (horizontal) দিকের edge শনাক্ত করে।

অধ্যায় ৭ — Convolution-এর ধাপে ধাপে সংখ্যাসহ হিসাব

Convolution-এর নিয়মটা এভাবে সংক্ষেপে লেখা যায়: filter-টাকে ইনপুটের উপরের-বাঁ কোণে বসাও, যে ঘরগুলো overlap করছে সেগুলো element-wise গুণ করে যোগ করো (এই যোগফলই feature map-এর একটা মান), এরপর filter-টাকে এক ঘর ডানে সরেও এবং একই প্রক্রিয়া চালাও; একটা সারির শেষে পৌঁছালে এক সারি নিচে নেমে বাঁ দিক থেকে আবার শুরু করো।

৭.১ গাণিতিক সংজ্ঞা

$$Z_{i,j} = \sum_{m=0}^{f-1} \sum_{n=0}^{f-1} X_{i+m, j+n} \cdot W_{m,n} + b$$

যেখানে,

প্রতীক	অর্থ
X	ইনপুট (input image বা feature map)
W	Filter / Kernel
b	Bias
f	Filter-এর আকার (filter size)
Z[i,j]	আউটপুট ফিচার ম্যাপের (i,j) অবস্থানের মান

৭.২ পূর্ণাঙ্গ সংখ্যাসহ উদাহরণ

ইনপুট ছবি X (4×4) — উপরের দুই সারি কালো (0), নিচের দুই সারি সাদা (255):

0	0	0	0
0	0	0	0
255	255	255	255
255	255	255	255

Filter W (3×3) — horizontal edge detector:

-1	-1	-1
0	0	0
1	1	1

bias $b = 0$ ধরে নিচ্ছি। এবার প্রতিটি বৈধ (valid) অবস্থানে হিসাব করা যাক।

অবস্থান (1,1) — X-এর উপরের-বাঁ 3×3 অংশ

ওভারল্যাপ করা অংশ: প্রথম তিন সারি, প্রথম তিন কলাম।

$$\begin{aligned}
 Z_{1,1} &= (0)(-1) + (0)(-1) + (0)(-1) + (0)(0) + (0)(0) + (0)(0) + (255)(1) + (255)(1) + (255)(1) \\
 &= 0 + 0 + 765 = 765
 \end{aligned}$$

অবস্থান (1,2) — এক ঘর ডানে সরে

ওভারল্যাপ করা কলামগুলো ভিন্ন হলেও মানগুলো একই (0,0,0 / 0,0,0 / 255,255,255) হওয়ায় ফলাফল একই থাকে:

$$Z_{1,2} = 765$$

অবস্থান (2,1) — এক সারি নিচে নেমে

এবার ওভারল্যাপ করা অংশ সারি ২,৩,৪ (0,0,0 / 255,255,255 / 255,255,255):

$$Z_{2,1} = (0)(-1) \times 3 + (255)(0) \times 3 + (255)(1) \times 3 = 0 + 0 + 765 = 765$$

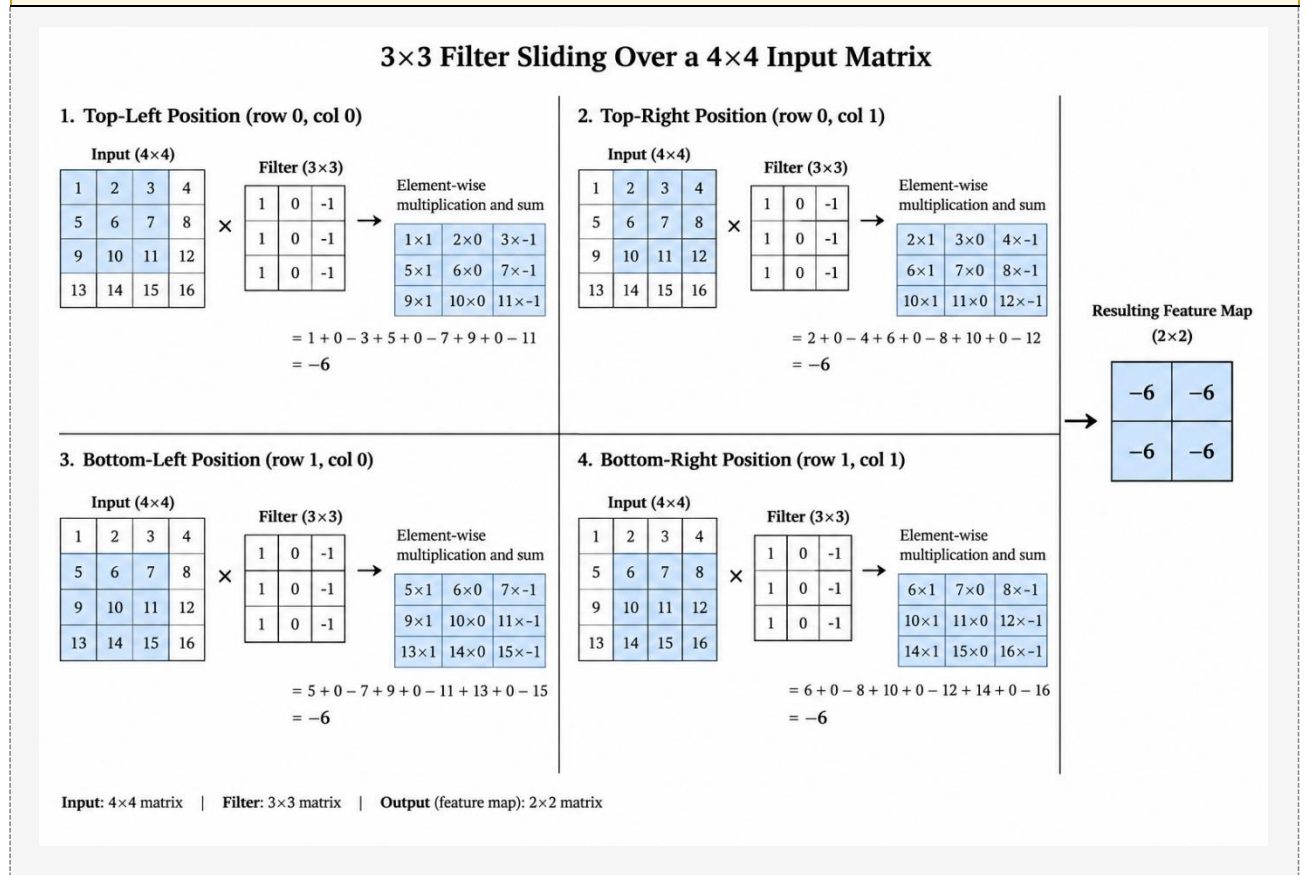
অবস্থান (2,2)

$$Z_{2,2} = 765$$

চূড়ান্ত Feature Map

765	765
765	765

মনে রাখো: বড় ধনাত্মক মান মানে ঐ অবস্থানে একটা শক্তিশালী আনুভূমিক edge আছে। যদি সম্পূর্ণ ইনপুট ছবি একই রঙের (সব ঘরে 255) হতো, তাহলে উত্তর হতো 0 — অর্থাৎ কোথাও কোনো edge নেই, কারণ intensity-তে কোনো পরিবর্তন নেই।



অধ্যায় ৮ — Feature Map, Bias এবং Activation Function

৮.১ Feature Map

Convolution operation-এর আউটপুটকে বলা হয় Feature Map (বা Activation Map)। এটি মূলত দেখায় — ইনপুট ছবির কোন কোন জায়গায় সেই filter-টা যে প্যাটার্ন খুঁজছে (যেমন vertical edge), সেটা কতটা শক্তিশালীভাবে উপস্থিত।

৮.২ Bias

Convolution-এর প্রতিটা আউটপুট মানে element-wise গুণ-যোগের পর একটা অতিরিক্ত trainable সংখ্যা bias (b) যোগ করা হয় — ঠিক যেমন ANN-এ প্রতিটা neuron-এর জন্য একটা bias থাকে। প্রতিটা filter-এর নিজস্ব একটাই bias থাকে (পুরো feature map জুড়ে একই bias ব্যবহৃত হয়, কারণ parameter sharing এখানেও প্রযোজ্য)।

৮.৩ Activation Function — ReLU

Convolution + Bias-এর পর সাধারণত একটা non-linear activation function প্রয়োগ করা হয়, যাতে নেটওয়ার্ক শুধু linear সম্পর্কের বাইরে জটিল (non-linear) প্যাটার্নও শিখতে পারে। CNN-এ সবচেয়ে বেশি ব্যবহৃত activation function হলো ReLU (Rectified Linear Unit):

$$ReLU(z) = \max(0, z)$$

অর্থাৎ, ঋণাত্মক মানকে 0 করে দেওয়া হয়, ধনাত্মক মান অপরিবর্তিত থাকে। এটি গণনাগতভাবে সহজ (computationally cheap) এবং vanishing gradient সমস্যা অনেকটা কমায় বলে গভীর নেটওয়ার্কে ব্যাপকভাবে ব্যবহৃত হয়।

অধ্যায় ৯ — Output Size-এর সূত্র

প্রতিবার convolution করলে আউটপুটের আকার ইনপুটের চেয়ে ছোট হয়ে যায় (padding না দিলে)। এর সাধারণ সূত্র:

$$n_{out} = n - f + 1$$

এখানে n = ইনপুটের আকার ($n \times n$ ধরে নেওয়া হচ্ছে), f = filter-এর আকার।

ইনপুট (n)	Filter (f)	হিসাব	আউটপুট
6×6	3×3	$6 - 3 + 1 = 4$	4×4
28×28	3×3	$28 - 3 + 1 = 26$	26×26

মনে রাখো: প্রতিটা convolution layer-এর পর ছবির আকার একটু একটু করে কমতে থাকে। গভীর নেটওয়ার্কে (অনেক layer) এভাবে চলতে থাকলে ছবি শেষে অতিরিক্ত ছোট হয়ে যেতে পারে — এই সমস্যার সমাধান হলো Padding, যা পরের অধ্যায়ে আলোচনা করা হবে।

অধ্যায় ১০ — Padding

১০.১ কেন Padding দরকার?

Padding ছাড়া convolution-এর দুইটা গুরুত্বপূর্ণ সমস্যা আছে:

সমস্যা ১ — Feature Map ক্রমাগত ছোট হতে থাকে

প্রতিটা convolution operation-এর পর ছবির আকার কমতে থাকে ($n \rightarrow n-f+1$)। যদি নেটওয়ার্কে অনেকগুলো (যেমন ৩০টা) convolution layer থাকে, তাহলে ছবি ক্রমাগত ছোট হতে হতে প্রায় বিলীন হয়ে যেতে পারে।

সমস্যা ২ — ছবির প্রান্তের (border) তথ্য কম গুরুত্ব পায়

একটা ইনপুট ছবিতে কোণার (corner) pixel filter দ্বারা মাত্র একবার স্পর্শ করা হয়, কিন্তু কেন্দ্রের (center) pixel filter দ্বারা বারবার (একাধিকবার) স্পর্শ করা হয়। ফলে কোণার/প্রান্তের pixel-এর অবদান output-এ তুলনামূলক কম থাকে, এবং ছবির প্রান্তবর্তী গুরুত্বপূর্ণ তথ্য হারিয়ে যাওয়ার ঝুঁকি থাকে।

১০.২ সমাধান — Zero Padding

সমাধান হলো মূল ছবির চারপাশে অতিরিক্ত একটা শূন্য-মূল্যের (0) বর্ডার যোগ করে দেওয়া, যাকে বলে Zero Padding। এতে কার্যকরী ইনপুটের আকার বেড়ে যায়, ফলে আউটপুটের আকার সংরক্ষিত থাকে এবং মূল ছবির প্রান্তের pixel-গুলো এখন আর প্রকৃত কোণায় থাকে না — সেগুলোও কেন্দ্রের pixel-এর মতোই একাধিকবার filter দ্বারা স্পর্শিত হয়।

১০.৩ Padding সহ Output Size-এর সূত্র

$$n_{out} = n + 2p - f + 1$$

এখানে p = padding-এর পরিমাণ, অর্থাৎ প্রতিটা পাশে (উপরে, নিচে, বামে, ডানে) কত স্তর শূন্য যোগ করা হচ্ছে। $2p$ ব্যবহার করা হয়েছে কারণ padding দুই পাশেই (উদাহরণস্বরূপ, উচ্চতার দিক থেকে উপর ও নিচ উভয় পাশে) সমানভাবে যোগ হয়।

উদাহরণ: ইনপুট 5×5 , filter 3×3 , $p = 1$ হলে —

$$n_{out} = 5 + 2(1) - 3 + 1 = 5$$

অর্থাৎ, আউটপুটের আকার একটুও কমল না — ইনপুটের সমান থেকে গেল।

১০.৪ Padding-এর দুইটা প্রচলিত ধরন (Keras-এর পরিভাষায়)

ধরন	p-এর মান	ফলাফল
"valid"	$p = 0$ (কোনো padding নেই)	আউটপুট ইনপুট থেকে ছোট হয়
"same"	নিচের সূত্র অনুযায়ী	আউটপুট = ইনপুট (আকার অপরিবর্তিত থাকে)

"same" padding-এর জন্য প্রয়োজনীয় p বের করতে, আমরা চাই $n_{out} = n$, অর্থাৎ:

$$n = n + 2p - f + 1 \Rightarrow 2p = f - 1$$

$p = (f - 1) / 2$	
Filter size (f)	"same" padding-এর জন্য p
3	1
5	2
7	3

মনে রাখো: এই কারণেই CNN-এ প্রায় সবসময় বিজোড় (odd) সাইজের filter (3, 5, 7) ব্যবহার করা হয় — জোড় সাইজে p ভগ্নাংশে পরিণত হয় (যেমন $f=4$ হলে $p=1.5$, যা অসম্ভব), এবং জোড় সাইজের filter-এর কোনো একক কেন্দ্রীয় (center) pixel-ও থাকে না।

Without Padding

1	2	3	4	5
6	7	8	9	10
11	12	13	14	15
16	17	18	19	20
21	22	23	24	25

Input (5×5)
(without padding)

With Padding $p = 1$, size 7×7

0	0	0	0	0	0	0
0	1	2	3	4	5	0
0	6	7	8	9	10	0
0	11	12	13	14	15	0
0	16	17	18	19	20	0
0	21	22	23	24	25	0
0	0	0	0	0	0	0

Padded Input (7×7)
(padding $p = 1$)

অধ্যায় ১১ — Stride

১১.১ সংজ্ঞা

Stride বলতে বোঝায় — প্রতিবার filter কত ঘর করে সরবে (লাফ দেবে)। ডিফল্ট stride = 1 মানে filter প্রতিবার ১ ঘর করে ডানে (এবং সারি শেষে ১ ঘর নিচে) সরে। stride = 2 হলে filter প্রতিবার ২ ঘর করে সরে, ফলে কিছু অবস্থান স্কিপ (এড়িয়ে) যায় এবং ফলে ফিচার ম্যাপ আরও ছোট হয়।

১১.২ সাধারণ সূত্র

Stride ছাড়া ($s=1$ ধরলে): $n - f + 1$ । সাধারণ stride s সহ:

$$n_{out} = \left\lfloor \frac{n - f}{s} \right\rfloor + 1$$

Padding ও Stride একসাথে ব্যবহার করলে সম্পূর্ণ সাধারণ সূত্র:

$$n_{out} = \left\lfloor \frac{n + 2p - f}{s} \right\rfloor + 1$$

১১.৩ Floor (⌊) কেন দরকার?

কখনো কখনো $(n+2p-f)/s$ একটা পূর্ণসংখ্যা নাও হতে পারে। উদাহরণস্বরূপ, ইনপুট উচ্চতা 6, filter 3, stride 2:

$$(6 - 3)/2 + 1 = 1.5 + 1 = 2.5$$

যেহেতু ২.৫টা আউটপুট-পজিশন বলে কিছু হয় না (আউটপুট-আকার অবশ্যই একটা পূর্ণসংখ্যা হতে হবে), তাই floor নেওয়া হয়:

$$\lfloor 1.5 \rfloor + 1 = 1 + 1 = 2$$

এর মানে হলো — শেষ দিকে filter সম্পূর্ণভাবে বসানোর জন্য যথেষ্ট pixel অবশিষ্ট নেই, তাই সেই অসম্পূর্ণ অবস্থানটুকু গণনায় বাদ (ignore) দেওয়া হয়।

১১.৪ Stride ব্যবহারের কারণ

কারণ	ব্যাখ্যা
High-level feature	বড় লাফ দিলে filter সূক্ষ্ম খুঁটিনাটি না ধরে তুলনামূলক বড়, উচ্চস্তরের গঠন (structure) ধরে
Computation কমানো	feature map ছোট হলে পরবর্তী layer-এ হিসাব ও মেমরি ব্যবহার কম হয়, ট্রেনিং দ্রুততর হয়

অধ্যায় ১২ — RGB / বহু-চ্যানেল ইনপুটে Convolution

রঙিন (RGB) ছবির ক্ষেত্রে filter-কেও 3D (একটি cuboid আকৃতির) হতে হয় — filter-এর depth অবশ্যই ইনপুটের channel সংখ্যার সমান হতে হবে।

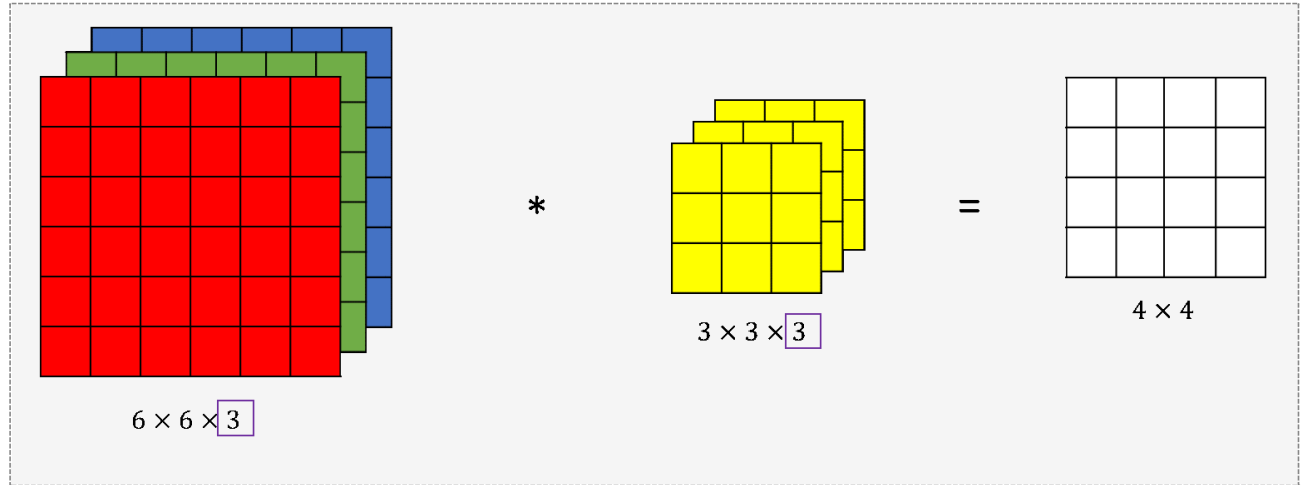
$$6 \times 6 \times 3 * 3 \times 3 \times 3 \rightarrow 4 \times 4$$

মনে রাখো: গুরুত্বপূর্ণ: আউটপুট কিন্তু 3D নয়, বরং 2D! কারণ তিনটা channel-এর গুণফল সবগুলো একসাথে যোগ হয়ে একটাই সংখ্যা তৈরি করে — channel-এর dimension-টা "যোগ হয়ে হারিয়ে যায়"।

গাণিতিক সংজ্ঞা:

$$Z_{i,j} = \sum_{c=1}^3 \sum_{m=0}^2 \sum_{n=0}^2 X_{i+m,j+n,c} \cdot W_{m,n,c} + b$$

এক convolution অবস্থানে মোট গুণ-যোগের সংখ্যা = $3 \times 3 \times 3 = 27$ টা গুণ, তারপর একটা bias যোগ।



অধ্যায় ১৩ — একাধিক Filter (Multiple Filters)

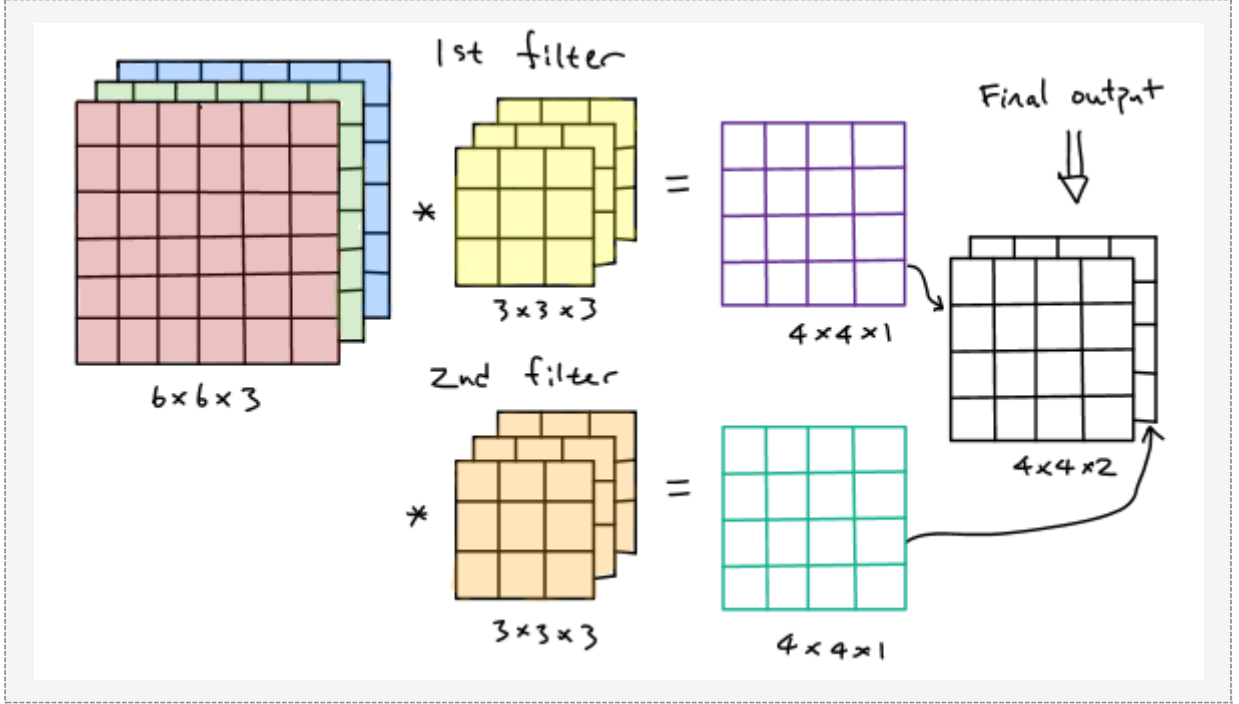
একটা মাত্র filter কেবল একটা নির্দিষ্ট feature ধরতে পারে (যেমন শুধু vertical edge)। বাস্তব সমস্যায় একসাথে অনেক ধরনের feature (vertical edge, horizontal edge, texture, রঙের প্যাটার্ন ইত্যাদি) দরকার হয়, তাই একাধিক filter ব্যবহার করা হয় — প্রতিটা filter আলাদা একটা feature শেখে।

$$[n \times n \times n_c (n - f + 1) \times (n - f + 1) \times k]$$

অর্থাৎ আউটপুট ভলিউমের depth নির্ধারিত হয় কতগুলো filter ব্যবহার করা হয়েছে তার সংখ্যা দ্বারা — প্রতিটা filter একটা করে 2D feature map তৈরি করে, এবং সবগুলো feature map একসাথে স্তরীভূত (stacked) হয়ে একটা আউটপুট ভলিউম গঠন করে।

উদাহরণ: $228 \times 228 \times 3$ ছবিতে 100টা $3 \times 3 \times 3$ filter প্রয়োগ করলে:

$$\text{আউটপুট} = 226 \times 226 \times 100 \quad [\text{যেহেতু } 228 - 3 + 1 = 226]$$



অধ্যায় ১৪ — Pooling

১৪.১ Convolution-এর পরেও সমস্যা কী থেকে যায়?

শুধু Convolution layer ব্যবহার করলেও দুইটা সমস্যা থেকে যায়:

সমস্যা ১ — Memory ও Computation

একাধিক filter ব্যবহার করলে আউটপুট ভলিউম খুব দ্রুত বিশাল হয়ে যেতে পারে। উদাহরণস্বরূপ, $228 \times 228 \times 3$ ছবিতে ১০০টা filter দিলে আউটপুট হয় $226 \times 226 \times 100$ — একটা মাত্র ছবির জন্যই প্রায় ৫১ লক্ষেরও বেশি সংখ্যা! ব্যাচে (batch) হাজার হাজার ছবি প্রসেস করতে গেলে memory ও computation দ্রুত ফুরিয়ে যায়।

সমস্যা ২ — Translation Variance

Convolution-এর feature detection অবস্থান-নির্ভর (location dependent) — একই বস্তু (যেমন একটা বিড়াল) ছবির বাঁ পাশে থাকলে এক রকম feature map তৈরি হয়, ডান পাশে সরে গেলে সম্পূর্ণ ভিন্ন feature map তৈরি হয়। অথচ বস্তুটা তো একই থেকে যাচ্ছে। এই সমস্যাকে বলে translation variance।

১৪.২ সমাধান — Down Sampling

Pooling হলো একটি down-sampling অপারেশন — feature map-এর প্রতিটা ছোট অঞ্চল (window) থেকে একটাই প্রতিনিধিত্বমূলক (representative) সংখ্যা রেখে বাকিটা ফেলে দেওয়া হয়। গুরুত্বপূর্ণ কথা: Pooling-এ কোনো trainable parameter (weight/bias) নেই — এটি নিছক একটা নির্ধারিত (fixed) গাণিতিক নিয়ম।

১৪.৩ Pooling-এর প্রকারভেদ

ধরন	নিয়ম
Max Pooling	প্রতিটা window থেকে সর্বোচ্চ মান নেওয়া হয়
Average Pooling	প্রতিটা window-এর মানগুলোর গড় নেওয়া হয়
Global Average / Max Pooling	পুরো একটা channel থেকে মাত্র একটা সংখ্যা (গড় বা সর্বোচ্চ) নেওয়া হয়

১৪.৪ Max Pooling — সংখ্যাসহ উদাহরণ

ইনপুট feature map (4×4), pool size = 2×2, stride = 2:

3	1	1	3
2	5	0	2
1	4	2	1
4	7	2	4

এই 4×4 ম্যাট্রিক্সকে চারটা 2×2 উইন্ডোতে ভাগ করা হলো (stride 2 হওয়ায় উইন্ডোগুলো একে অপরের সাথে overlap করে না):

উইন্ডো (অবস্থান)	মানসমূহ	Max
বাঁ-উপর	3, 1, 2, 5	5
ডান-উপর	1, 3, 0, 2	3
বাঁ-নিচ	1, 4, 4, 7	7
ডান-নিচ	2, 1, 2, 4	4

ফলাফল (2×2 Feature Map):

5	3
7	4

গাণিতিক সংজ্ঞা:

$$P_{i,j} = \max_{0 \leq m, n < k} A_{i \cdot s + m, j \cdot s + n}$$

এখানে k = pool size, s = stride।

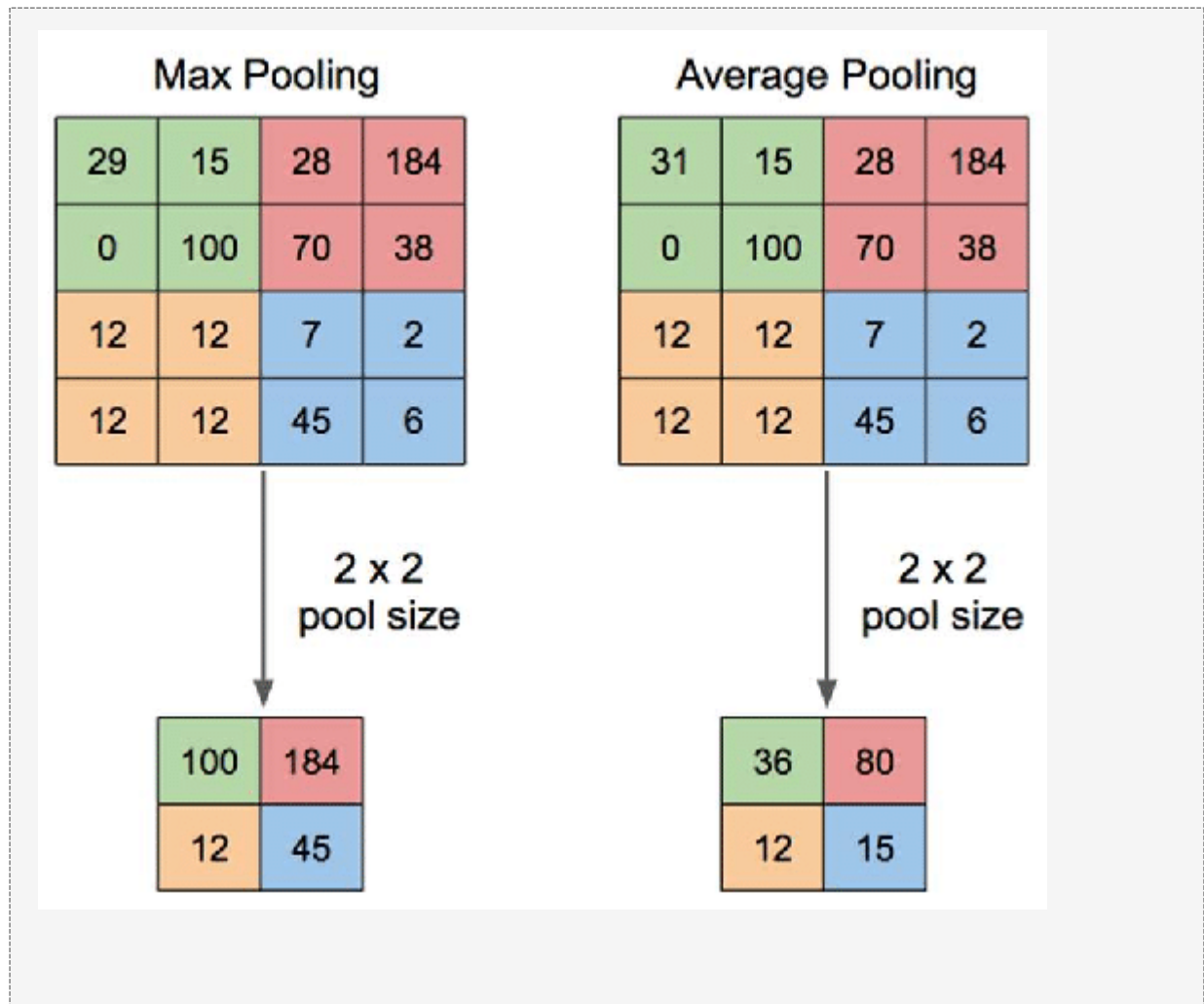
Average Pooling হলে একই সূত্র হবে (উদাহরণে বাঁ-উপরের ব্লকে $(3+1+2+5)/4 = 2.75$):

$$P_{i,j} = \frac{1}{k^2} \sum_{m=0}^{k-1} \sum_{n=0}^{k-1} A_{i \cdot s + m, j \cdot s + n}$$

১৪.৫ Pooling-এর Output Size

$$n_{\text{out}} = \left\lfloor \frac{n - k}{s} \right\rfloor + 1$$

উদাহরণ: 4×4 ইনপুট, $k=2$, $s=2 \rightarrow (4-2)/2 + 1 = 2$, অর্থাৎ 2×2 আউটপুট — উপরের উদাহরণের সাথে মিলে যাচ্ছে।



১৪.৬ Pooling বহু-চ্যানেল (Volume) Feature Map-এর উপর

Pooling প্রতিটা channel-এ আলাদা আলাদাভাবে (independently) প্রয়োগ করা হয়। ফলে depth (channel সংখ্যা) অপরিবর্তিত থাকে — শুধু উচ্চতা ও প্রস্থ কমে।

$$226 \times 226 \times 100 \rightarrow \text{Max Pool} (2 \times 2, \text{stride } 2) \rightarrow 113 \times 113 \times 100$$

হিসাব: $(226-2)/2 + 1 = 113$ । আকার প্রায় ৪ গুণ কমল, কিন্তু channel সংখ্যা একই রইল।

১৪.৭ Translation Invariance — একটি সত্য ও সঠিক ব্যাখ্যা

Max Pooling-এর একটা গুরুত্বপূর্ণ উপকারিতা হলো এটি নেটওয়ার্ককে ছোটখাটো স্থানান্তরের (small translation) প্রতি কিছুটা সহনশীল (tolerant/robust) করে তোলে। উদাহরণস্বরূপ, একটা 2×2 উইন্ডোর ভেতরে সর্বোচ্চ মান যদি একটু এদিক-ওদিক সরে যায়, তাও সেই একই উইন্ডোর max ফলাফল অপরিবর্তিত থাকতে পারে।

মনে রাখো: তবে এখানে বৈজ্ঞানিকভাবে সত্য থাকা জরুরি: Pooling "পারফেক্ট" বা "গ্যারান্টিযুক্ত" translation invariance দেয় না। এটি শুধুমাত্র ছোট, স্থানীয় (local) স্থানান্তরের প্রতি আংশিক সহনশীলতা (approximate tolerance) প্রদান করে — এবং তাও শুধুমাত্র একটা পুলিং উইন্ডোর ভেতরের সীমার মধ্যে। বস্তু যদি উইন্ডোর সীমা পেরিয়ে অনেক দূরে সরে যায়, তাহলে ফলাফল বদলে যেতে পারে। প্রকৃত translation invariance (approximate translation tolerance) নেটওয়ার্কের multiple layer-এর সমন্বিত প্রভাবে ধীরে ধীরে তৈরি হয়।

১৪.৮ Pooling-এর সুবিধা (Advantages)

সুবিধা	ব্যাখ্যা
Reduce Size	বিশাল ভলিউম ধাপে ধাপে ছোট হয়ে আসে, মেমরি ও কম্পিউটেশন সাশ্রয় হয়
Translation Tolerance	বস্তুর সামান্য স্থানান্তরে ফলাফল মোটামুটি স্থিতিশীল থাকে (উপরে বর্ণিত সীমাবদ্ধতাসহ)
Enhanced Feature (শুধু Max Pooling)	সবচেয়ে শক্তিশালী activation-টাই টিকে থাকে, দুর্বল সাড়াগুলো বাদ পড়ে
কোনো Training দরকার নেই	কোনো trainable parameter না থাকায় মডেলের ভার (parameter count) বাড়ায় না

১৪.৯ Pooling-এর অসুবিধা (Disadvantages)

অসুবিধা	ব্যাখ্যা
Segmentation-এ ক্ষতিকর	যেসব কাজে প্রতিটা pixel-এর নির্দিষ্ট অবস্থান জানা জরুরি (যেমন Image Segmentation), সেখানে pooling সেই সূক্ষ্ম অবস্থানগত তথ্য নষ্ট করে ফেলে
তথ্য হারানো (Information Loss)	একটা উইন্ডোর একাধিক মান থেকে মাত্র একটা মান রেখে বাকিগুলো চিরতরে ফেলে দেওয়া হয়, যা আর ফিরে পাওয়া যায় না

মনে রাখো: Max Pooling ঋণাত্মক (negative) মানের ক্ষেত্রেও কিছুটা সতর্কতা দাবি করে — এটি সবচেয়ে "কম ঋণাত্মক" মানটাই বেছে নেয়, যা সবসময় সবচেয়ে অর্থবহ নাও হতে পারে। তবে যেহেতু বাস্তবে ReLU-এর পরেই Pooling ব্যবহার করা হয় (এবং ReLU সব ঋণাত্মক মানকে 0 করে দেয়), তাই বাস্তবে এই সমস্যাটা সাধারণত দেখা যায় না।

১৪.১০ Global Pooling

Global Pooling-এ পুরো একটা channel-এর সব মান থেকে মাত্র একটাই সংখ্যা বের করা হয়:

$$7 \times 7 \times 512 \rightarrow \text{Global Average Pooling} \rightarrow 1 \times 1 \times 512$$

Global Average Pooling প্রতি channel-এর গড় নেয়, আর Global Max Pooling প্রতি channel-এর সর্বোচ্চ মান নেয়। এটি প্রায়ই Flatten layer-এর বিকল্প হিসেবে ব্যবহৃত হয়, কারণ Flatten করলে বিশাল সংখ্যক প্যারামিটার তৈরি হয়ে যায়; Global Pooling সেই বিস্ফোরণ ঠেকায় এবং overfitting কমাতে সাহায্য করে।

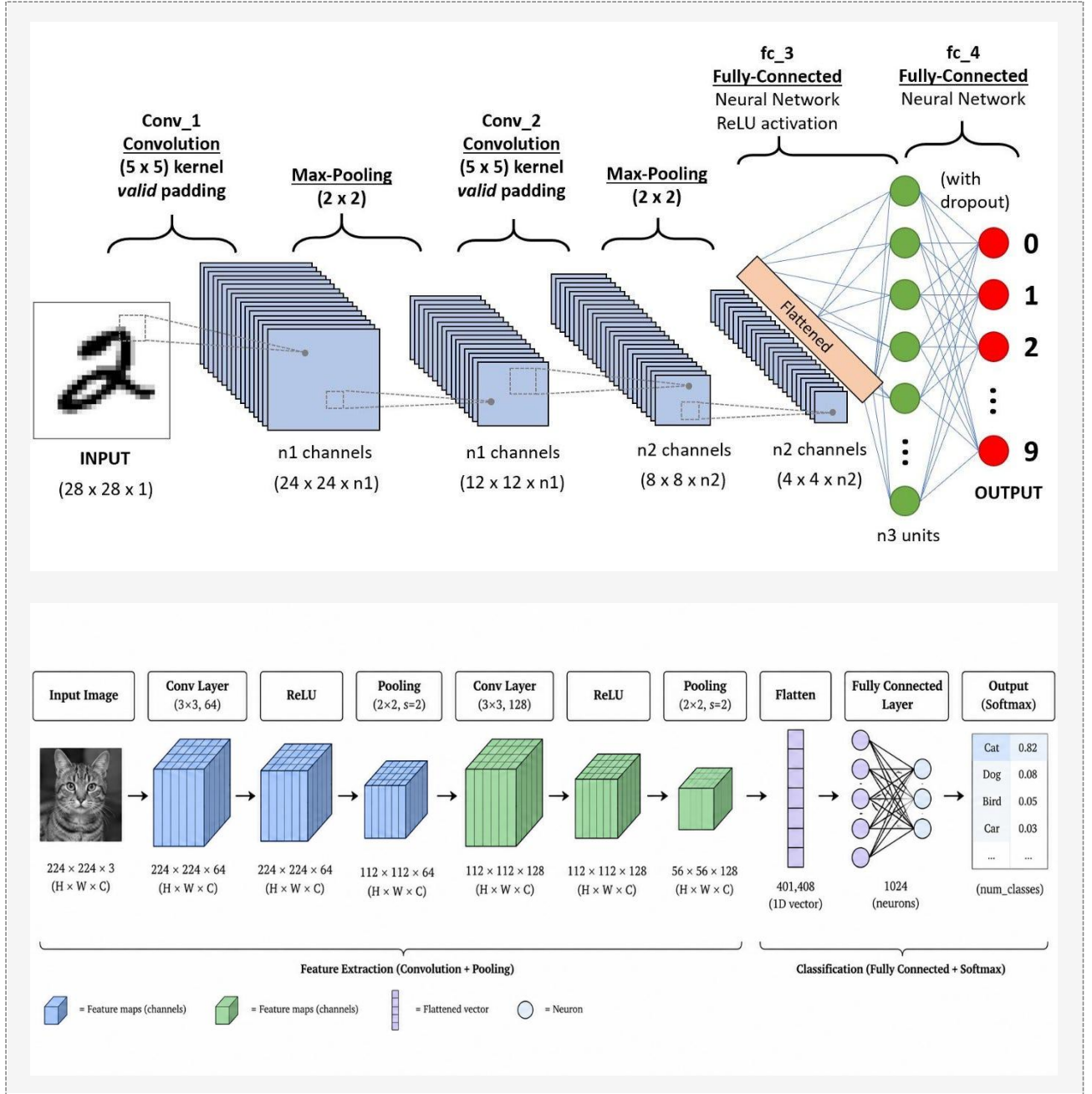
অধ্যায় ১৫ — CNN Architecture

একটা সাধারণ CNN আর্কিটেকচার সাধারণত নিম্নলিখিত প্যাটার্ন অনুসরণ করে:

$$\text{Input} \rightarrow [\text{Conv} \rightarrow \text{Activation} \rightarrow \text{Pooling}] \times N \rightarrow \text{Flatten} / \text{Global Pooling} \rightarrow \text{Fully Connected} \rightarrow \text{Output}$$

এখানে $[\text{Conv} \rightarrow \text{Activation} \rightarrow \text{Pooling}]$ ব্লকটি বেশ কয়েকবার (N বার) পুনরাবৃত্তি হয়। এই কাঠামোর পেছনে দুটি গুরুত্বপূর্ণ ডিজাইন নীতি কাজ করে:

প্রবণতা	কারণ
স্থানিক (spatial) আকার ধীরে ধীরে কমে (উচ্চতা-প্রস্থ)	Pooling ও stride-এর প্রভাবে, যা computation কমায় এবং higher-level feature-এ ফোকাস করতে সাহায্য করে
Channel/feature সংখ্যা ধীরে ধীরে বাড়ে	প্রতিটা layer-এ বেশি সংখ্যক filter ব্যবহার করে আরও বেশি ধরনের complex feature শেখার সুযোগ তৈরি করা হয়



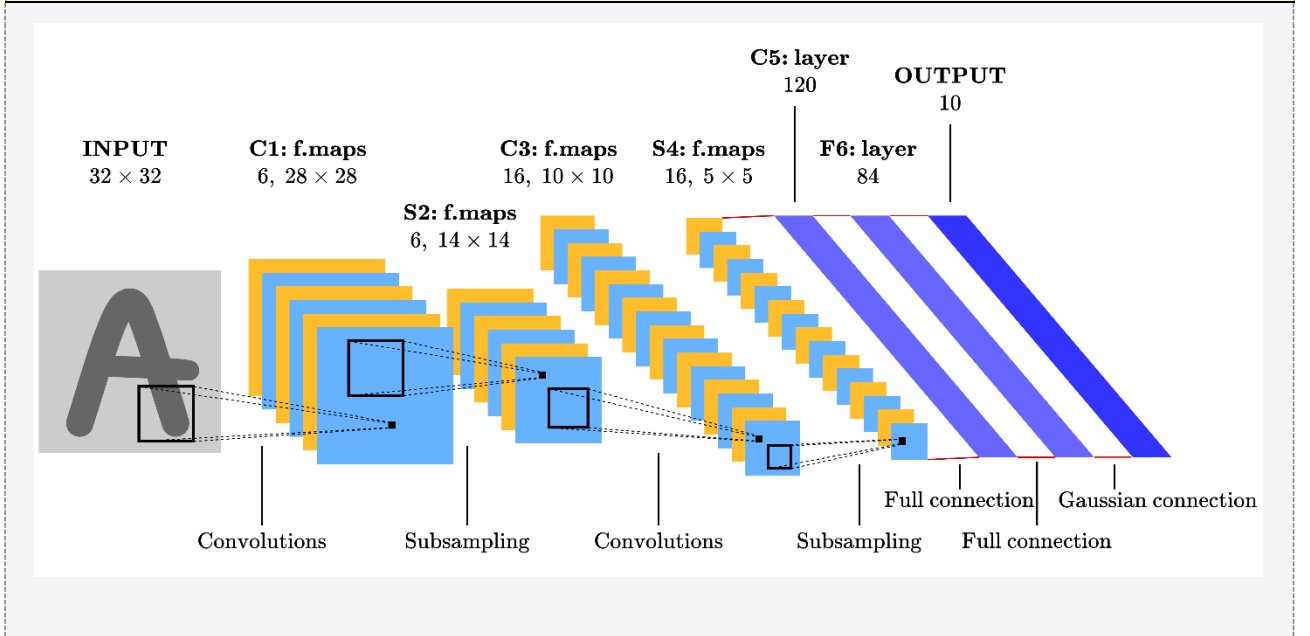
১৫.১ LeNet-5 — একটি সম্পূর্ণ ক্লাসিক উদাহরণ

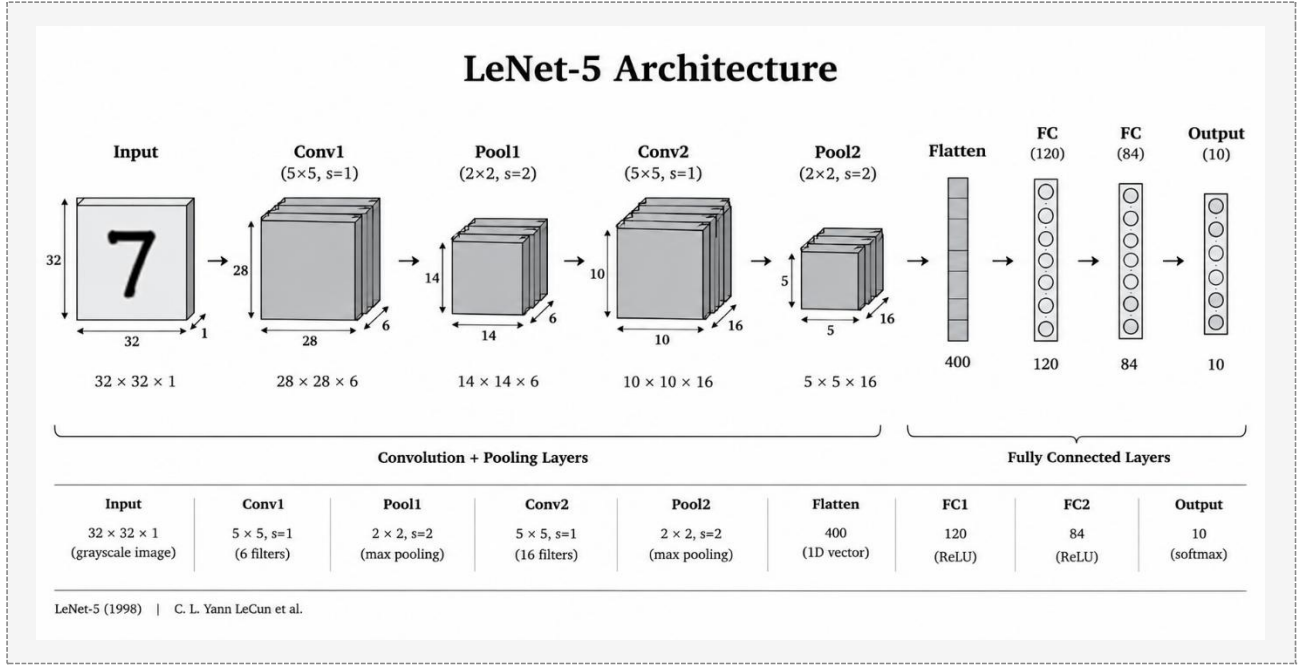
LeNet-5 (Yann LeCun, ১৯৯৮) হাতের লেখা সংখ্যা (digit, 0–9) চেনার জন্য ডিজাইন করা প্রথম সফল CNN আর্কিটেকচারগুলোর একটি। এর সম্পূর্ণ স্তর-ভিত্তিক গঠন এবং প্রতিটা স্তরে আকার কীভাবে পরিবর্তিত হয়, তা নিচে হিসাবসহ দেখানো হলো:

Layer	Operation	হিসাব	আউটপুট আকার
Input	—	—	$32 \times 32 \times 1$
Conv 1	৬টা filter, 5×5 , stride 1	$32 - 5 + 1 = 28$	$28 \times 28 \times 6$
Pool 1	Max Pool 2×2 , stride 2	$(28 - 2) / 2 + 1 = 14$	$14 \times 14 \times 6$

Layer	Operation	হিসাব	আউটপুট আকার
Conv 2	16টা filter, 5×5 , stride 1	$14 - 5 + 1 = 10$	$10 \times 10 \times 16$
Pool 2	Max Pool 2×2 , stride 2	$(10 - 2) / 2 + 1 = 5$	$5 \times 5 \times 16$
Flatten	$2D \rightarrow 1D$	$5 \times 5 \times 16$	400
FC 1	Fully Connected	—	120
FC 2	Fully Connected	—	84
Output	Softmax (10 ক্লাস)	—	10

মনে রাখো: লক্ষণীয়: উচ্চতা-প্রস্থ ক্রমাগত কমছে ($32 \rightarrow 28 \rightarrow 14 \rightarrow 10 \rightarrow 5$), অথচ depth (channel) বাড়ছে ($1 \rightarrow 6 \rightarrow 16$)। এটাই CNN-এর সাধারণ নকশা নীতির একটা বাস্তব উদাহরণ — জায়গার তথ্য কমিয়ে ফিচারের বৈচিত্র্য বাড়ানো।





১৫.২ বিখ্যাত CNN আর্কিটেকচারসমূহ (সংক্ষিপ্ত)

আর্কিটেকচার	বৈশিষ্ট্য
LeNet	Yann LeCun-এর তৈরি প্রথম সফল CNN, digit recognition-এ ব্যবহৃত
AlexNet (2012)	ImageNet প্রতিযোগিতার বিজয়ী; ReLU activation, Dropout, ও GPU-ভিত্তিক training জনপ্রিয় করে তোলে
VGGNet	সরল ও গভীর গঠন — পুরো নেটওয়ার্ক জুড়ে শুধুমাত্র 3×3 filter ব্যবহার
GoogLeNet / Inception	Inception module ব্যবহার করে — একই layer-এ বিভিন্ন আকারের filter (1×1, 3×3, 5×5) সমান্তরালে চালানো হয়
ResNet	Skip connection / residual connection চালু করে, যা অত্যন্ত গভীর নেটওয়ার্ক (শত শত layer) প্রশিক্ষণযোগ্য করে তোলে

অধ্যায় ১৬ — CNN বনাম ANN

১৬.১ মিল (Similarities)

দিক	ব্যাখ্যা
মৌলিক একক	দুটোতেই neuron, weight, bias থাকে
Forward Propagation	input → weight দিয়ে গুণ → bias যোগ → activation, এই মূল ধাপ দুটোতেই অভিন্ন
Activation Function	দুটোতেই ReLU, Sigmoid, Softmax ইত্যাদি ব্যবহার হয়
Loss Function	দুটোতেই Cross-Entropy, MSE ইত্যাদি ব্যবহৃত হয়
Backpropagation	দুটোতেই Chain Rule দিয়ে gradient বের করা হয়

দিক	ব্যাখ্যা
Gradient Descent	দুটোতেই একই নিয়মে weight আপডেট হয়
শেষ অংশ	CNN-এর একেবারে শেষের Fully Connected layer আসলে একটা সাধারণ ANN-ই

১৬.২ পার্থক্য (Differences)

দিক	ANN	CNN
Input	1D vector (flatten করা)	2D/3D ছবি (spatial গঠন অক্ষত)
সংযোগ (Connectivity)	Fully connected — প্রতিটা neuron সব ইনপুটের সাথে যুক্ত	Sparse/Local — filter শুধু একটা ছোট local অঞ্চল দেখে
Weight	প্রতিটা সংযোগের আলাদা weight	Parameter Sharing — একই filter পুরো ছবিতে পুনর্ব্যবহৃত হয়
প্যারামিটার সংখ্যা	সাধারণত বিশাল	তুলনামূলকভাবে অনেক কম
Spatial তথ্য	flatten করায় নষ্ট হয়ে যায়	Convolution layer-এ সংরক্ষিত থাকে
Translation-এর প্রতি সংবেদনশীলতা	সামান্য স্থানান্তরেও কর্মক্ষমতা ভেঙে পড়তে পারে	Pooling-এর কারণে তুলনামূলক সহনশীল
সাধারণ Layer টাইপ	শুধুমাত্র Dense/FC	Conv + Pool + Dense-এর সমন্বয়
সাধারণ ব্যবহার	Tabular / structured data	Image, video, spatial data

সংখ্যাসহ উদাহরণ: একটা $228 \times 228 \times 3$ ছবিকে ANN-এ 1000 neuron-এর প্রথম hidden layer-এ দিলে প্যারামিটার সংখ্যা দাঁড়ায় প্রায় $155952 \times 1000 \approx 1.56 \times 10^8$ কোটি ৬০ লক্ষ। অথচ CNN-এ একটা $3 \times 3 \times 3$ filter-এর প্যারামিটার সংখ্যা মাত্র 28 (27টা weight + 1টা bias)।

অধ্যায় ১৭ — Trainable Parameters গণনা

একটা Convolution layer-এর মোট trainable parameter সংখ্যা বের করার সূত্র:

$$\text{Parameters} = (f_h \times f_w \times C_{in} + 1) \times C_{out}$$

এখানে $f_h \times f_w$ = filter-এর উচ্চতা $\times$ প্রস্থ, C_{in} = ইনপুটের channel সংখ্যা, C_{out} = ব্যবহৃত filter-এর সংখ্যা (অর্থাৎ আউটপুটের channel সংখ্যা)। এখানে "+1" অংশটি প্রতিটা filter-এর নিজস্ব bias-কে প্রতিনিধিত্ব করে — প্রতিটা filter-এর জন্য একটামাত্র bias থাকে (parameter sharing-এর কারণে)।

উদাহরণ: ইনপুট $228 \times 228 \times 3$, filter 3×3 , filter সংখ্যা 100 —

$$\text{Parameters} = (3 \times 3 \times 3 + 1) \times 100 = 28 \times 100 = 2800$$

এই মাত্র 2800টা প্যারামিটার দিয়েই CNN $226 \times 226 \times 100$ আকারের একটা সমৃদ্ধ feature ভলিউম তৈরি করে ফেলতে পারে — যেখানে সমতুল্য একটা fully-connected ANN layer-এ লাগত কোটি কোটি প্যারামিটার। এটাই CNN-কে parameter-efficient করে তোলে।

অধ্যায় ১৮ — CNN-এর সম্পূর্ণ Forward Propagation

এই অধ্যায় থেকে শুরু করে বাকি পুরো ডকুমেন্ট জুড়ে আমরা একটা ছোট, সম্পূর্ণ CNN নিয়ে কাজ করব, যাতে প্রতিটা গাণিতিক ধাপ হাতে-কলমে যাচাই করা যায়। নেটওয়ার্কটির গঠন:

$$X \rightarrow \text{Convolution} \rightarrow \text{ReLU} \rightarrow \text{Max Pooling} \rightarrow \text{Flatten} \rightarrow \text{Fully Connected} \rightarrow \text{Sigmoid} \rightarrow \text{Loss}$$

১৮.১ নোটেশন (Notation)

সম্পূর্ণ ডকুমেন্ট জুড়ে নিচের নোটেশন সামঞ্জস্যপূর্ণভাবে ব্যবহার করা হবে:

প্রতীক	অর্থ
X	ইনপুট (input image)
W_1, b_1	Convolution layer-এর filter ও bias
Z_1	Convolution-এর আউটপুট (activation প্রয়োগের আগে)
A_1	ReLU প্রয়োগের পরের আউটপুট
P_1	Max Pooling-এর আউটপুট
F	Flatten করার পর 1D vector
W_2, b_2	Fully Connected (FC) layer-এর weight ও bias
Z_2	FC layer-এর আউটপুট (activation প্রয়োগের আগে)
A_2	Sigmoid প্রয়োগের পরের আউটপুট (চূড়ান্ত prediction, $\hat{y}$)
L	Loss (ক্ষতি)

১৮.২ Forward Propagation সমীকরণসমূহ

$$Z_1 = \text{Conv}(X, W_1) + b_1$$

$$A_1 = \text{ReLU}(Z_1)$$

$$P_1 = \text{MaxPool}(A_1)$$

$$F = \text{Flatten}(P_1)$$

$$Z_2 = W_2 F + b_2$$

$$A_2 = \sigma(Z_2)$$

$$L = \text{BCE}(y, A_2)$$

প্রতিটা ধাপের ব্যাখ্যা:

ধাপ	কী ঘটছে
$Z_1 = \text{Conv}(X, W_1) + b_1$	Filter ছবির উপর স্লাইড করে local feature বের করে একটা feature map তৈরি করে
$A_1 = \text{ReLU}(Z_1)$	ঋণাত্মক মানগুলোকে শূন্য করে non-linearity যোগ করা হয়
$P_1 = \text{MaxPool}(A_1)$	প্রতিটা ছোট উইন্ডো থেকে সর্বোচ্চ মান নিয়ে আকার কমানো হয়
$F = \text{Flatten}(P_1)$	2D ম্যাট্রিক্সকে সোজা করে একটা 1D vector-এ রূপান্তর করা হয়
$Z_2 = W_2 F + b_2$	Fully Connected layer — সাধারণ ANN-এর মতো weighted sum
$A_2 = \sigma(Z_2)$	Sigmoid দিয়ে ফলাফলকে 0 থেকে 1-এর মধ্যে একটা probability-তে রূপান্তর করা হয়

এখানে ব্যবহৃত ফাংশনগুলো:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

$$\text{ReLU}(z) = \max(0, z)$$

১৮.৩ সম্পূর্ণ সংখ্যাসহ Forward Propagation উদাহরণ

ইনপুট X (4×4):

1	2	0	1
0	1	3	2
2	0	1	0
1	1	2	3

Filter W_1 (3×3), bias $b_1 = 0$:

1	0	-1
1	0	-1
1	0	-1

Convolution-এর আউটপুট Z_1 (2×2) হিসাব [$n_{\text{out}} = 4 - 3 + 1 = 2$]:

$$Z_{1,1} = (1 \cdot 1 + 2 \cdot 0 + 0 \cdot (-1)) + (0 \cdot 1 + 1 \cdot 0 + 3 \cdot (-1)) + (2 \cdot 1 + 0 \cdot 0 + 1 \cdot (-1)) = (1-0) + (0-3) + (2-1) = 1 - 3 + 1 = -1$$

$$Z_{1,2} = (2 \cdot 1 + 0 \cdot 0 + 1 \cdot (-1)) + (1 \cdot 1 + 3 \cdot 0 + 2 \cdot (-1)) + (0 \cdot 1 + 1 \cdot 0 + 0 \cdot (-1)) = (2-1) + (1-2) + (0-0) = 1 - 1 + 0 = 0$$

$$Z_{2,1} = (0 \cdot 1 + 1 \cdot 0 + 3 \cdot (-1)) + (2 \cdot 1 + 0 \cdot 0 + 1 \cdot (-1)) + (1 \cdot 1 + 1 \cdot 0 + 2 \cdot (-1)) = (0-3) + (2-1) + (1-2) = -3 + 1 - 1 = -3$$

$$Z_{2,2} = (1 \cdot 1 + 3 \cdot 0 + 2 \cdot (-1)) + (0 \cdot 1 + 1 \cdot 0 + 0 \cdot (-1)) + (1 \cdot 1 + 2 \cdot 0 + 3 \cdot (-1)) = (1-2) + (0-0) + (1-3) = -1 + 0 - 2 = -3$$

সুতরাং:

-1	0
-3	-3

এবার ReLU প্রয়োগ করি — ঋণাত্মক মান শূন্য হয়ে যাবে:

0	0
0	0

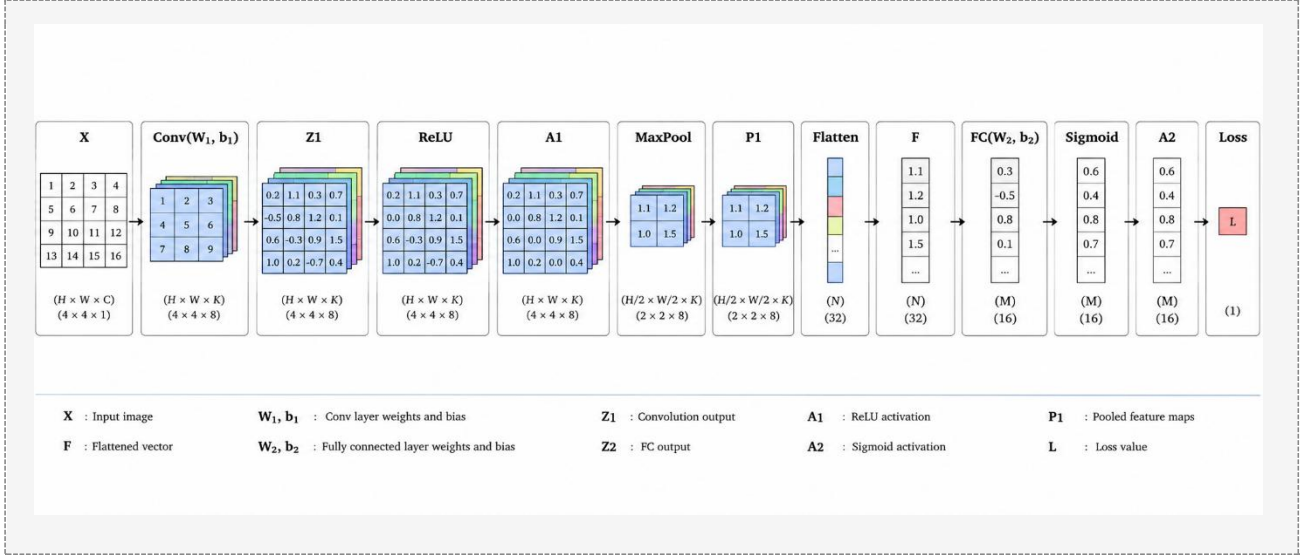
মনে রাখো: এই নির্দিষ্ট উদাহরণে সবগুলো Z_1 মান ঋণাত্মক বা শূন্য হওয়ায় A_1 পুরোপুরি শূন্য ম্যাট্রিক্স হয়ে গেছে। বাস্তব প্রশিক্ষণে filter-এর weight ও bias প্রশিক্ষণের মাধ্যমে এমনভাবে সামঞ্জস্য করা হয় যাতে অর্থবহ (non-trivial) আউটপুট তৈরি হয়। পরের সেকশনে (সম্পূর্ণ numerical backpropagation) আমরা এমন একটা উদাহরণ ব্যবহার করব যেখানে non-zero gradient প্রবাহ স্পষ্টভাবে দেখানো যায়।

১৮.৪ Max Pooling, Flatten, এবং FC (সাধারণ ধারাবাহিকতা)

A_1 (2×2) থেকে Max Pooling করলে (pool size = আউটপুটের সমান হলে পুরো ম্যাট্রিক্স থেকে একটাই সর্বোচ্চ মান পাওয়া যাবে) P_1 পাওয়া যায়। এরপর Flatten করে P_1 -কে একটা 1D vector F -এ রূপান্তর করা হয়। তারপর FC layer-এর মাধ্যমে:

$$Z_2 = W_2 F + b_2, \quad A_2 = \sigma(Z_2)$$

A_2 হলো চূড়ান্ত prediction (0 থেকে 1-এর মধ্যে একটা সংখ্যা), যা প্রকৃত label y -এর সাথে তুলনা করে Loss গণনা করা হয়। পরের অধ্যায়ে Loss Function এবং তারপর সম্পূর্ণ numerical উদাহরণে (non-zero মান দিয়ে) এই পুরো প্রক্রিয়া বিস্তারিতভাবে দেখানো হবে।



অধ্যায় ১৯ — Loss Function ও Gradient Descent

১৯.১ Loss Function — Binary Cross Entropy

এই উদাহরণ নেটওয়ার্কটি একটা Binary Classification সমস্যা সমাধান করছে (যেমন: ছবিতে বিড়াল আছে কি নেই)। এই ধরনের সমস্যায় সবচেয়ে বহুল ব্যবহৃত loss function হলো Binary Cross Entropy (BCE):

$$L = -y \log(A_2) - (1 - y) \log(1 - A_2)$$

এখানে y = প্রকৃত label (0 অথবা 1), A_2 = নেটওয়ার্কের predicted probability। এই loss function-টি এমনভাবে ডিজাইন করা, যাতে prediction যত ভুল হবে, loss তত বেশি বড় হবে; আর prediction সঠিক হলে loss শূন্যের কাছাকাছি থাকবে।

১৯.২ Gradient Descent

Gradient Descent হলো একটা অপটিমাইজেশন পদ্ধতি, যাদিয়ে নেটওয়ার্কের সব trainable parameter (W_1, b_1, W_2, b_2) ধীরে ধীরে এমনভাবে আপডেট করা হয়, যাতে Loss কমতে থাকে। প্রতিটা প্যারামিটার একই সাধারণ নিয়মে আপডেট হয়:

$$W_1 \leftarrow W_1 - \eta \frac{\partial L}{\partial W_1}$$

$$b_1 \leftarrow b_1 - \eta \frac{\partial L}{\partial b_1}$$

$$W_2 \leftarrow W_2 - \eta \frac{\partial L}{\partial W_2}$$

$$b_2 \leftarrow b_2 - \eta \frac{\partial L}{\partial b_2}$$

এখানে η (eta) = learning rate, একটা ছোট ধনাত্মক hyperparameter যা প্রতিবার আপডেটের "পদক্ষেপের আকার" নিয়ন্ত্রণ করে।

মনে রাখো: $\partial L / \partial W$ মানে কী? Loss L , W -এর মানের সামান্য পরিবর্তনে কতটা পরিবর্তিত হবে, সেটাই $\partial L / \partial W$ (Loss-এর W -এর সাপেক্ষে partial derivative বা gradient)। এই gradient-টাই আমাদের বলে দেয় কোন দিকে W -কে সরালে Loss কমবে। Gradient Descent-এর আসল কাজ হলো এই চারটা derivative ($\partial L / \partial W_1$, $\partial L / \partial b_1$, $\partial L / \partial W_2$, $\partial L / \partial b_2$) বের করা — আর সেই কাজটাই করে Backpropagation, যা পরবর্তী অধ্যায়ের মূল বিষয়।

অধ্যায় ২০ — Backpropagation: ধারণা ও কৌশল

২০.১ মূল অন্তর্দৃষ্টি — Forward বনাম Backward

দিক	প্রবাহের দিক
Forward Propagation	Input → Prediction → Loss (বাম থেকে ডানে)
Backpropagation	Loss → Error Signal → Gradient → Parameter Update (ডান থেকে বামে)

Forward Propagation-এ আমরা ইনপুট থেকে শুরু করে একের পর এক ধাপ পার হয়ে একটা prediction ও তারপর Loss বের করি। Backpropagation-এ আমরা উল্টো দিকে যাই — Loss থেকে শুরু করে, Chain Rule ব্যবহার করে ধাপে ধাপে পেছনের দিকে প্রতিটা প্যারামিটারের সাপেক্ষে gradient বের করে আনি। প্রতিটা layer একটা করে "error signal" পেছনের layer-এর দিকে পাঠায়।

২০.২ কৌশল — নেটওয়ার্ককে দুই ভাগে ভাগ করা

আমাদের নেটওয়ার্ককে দুইটা যৌক্তিক অংশে ভাগ করে গণনা করা সবচেয়ে সহজ:

অংশ	কভার করে	প্যারামিটার
ANN অংশ	Flatten থেকে Output পর্যন্ত ($F \rightarrow Z_2 \rightarrow A_2 \rightarrow L$)	W_2, b_2
CNN অংশ	Input থেকে Flatten পর্যন্ত ($X \rightarrow Z_1 \rightarrow A_1 \rightarrow P_1 \rightarrow F$)	W_1, b_1

প্রথমে আমরা ANN অংশের gradient বের করব ($\partial L/\partial W_2$, $\partial L/\partial b_2$)। তারপর সেই একই gradient-প্রবাহকে (বিশেষত $\partial L/\partial F$) পেছনের দিকে আরও প্রবাহিত করে CNN অংশে (Flatten → Pooling → ReLU → Convolution) নিয়ে যাব।

অধ্যায় ২১ — Backpropagation পর্বক: ANN অংশ (W_2 , b_2)

২১.১ Chain Rule সেটআপ

আমরা $\partial L/\partial W_2$ বের করতে চাই — অর্থাৎ, W_2 -এর সামান্য পরিবর্তনে Loss কতটা বদলাবে। যেহেতু W_2 , Z_2 -এর মাধ্যমে, এবং Z_2 , A_2 -এর মাধ্যমে, এবং A_2 সরাসরি Loss-কে প্রভাবিত করে, তাই Chain Rule প্রয়োগ করে আমরা এই সম্পর্কটাকে কয়েকটা ছোট, সহজে গণনাযোগ্য অংশে ভাঙতে পারি:

$$\frac{\partial L}{\partial W_2} = \frac{\partial L}{\partial A_2} \cdot \frac{\partial A_2}{\partial Z_2} \cdot \frac{\partial Z_2}{\partial W_2}$$

প্রতিটা টার্মের অর্থ:

টার্ম	অর্থ
$\partial L/\partial A_2$	A_2 -এর সামান্য পরিবর্তনে Loss কতটা বদলায়
$\partial A_2/\partial Z_2$	Z_2 -এর সামান্য পরিবর্তনে A_2 (sigmoid-এর আউটপুট) কতটা বদলায়
$\partial Z_2/\partial W_2$	W_2 -এর সামান্য পরিবর্তনে Z_2 কতটা বদলায়

২১.২ প্রথম টার্ম: $\frac{\partial L}{\partial A_2}$

Binary Cross Entropy Loss:

$$L = -y \log(A_2) - (1 - y) \log(1 - A_2)$$

A_2 -এর সাপেক্ষে differentiate করলে:

$$\frac{\partial L}{\partial A_2} = -\frac{y}{A_2} + \frac{1 - y}{1 - A_2}$$

Common denominator নিলে:

$$\frac{\partial L}{\partial A_2} = \frac{-y(1 - A_2) + A_2(1 - y)}{A_2(1 - A_2)}$$

Numerator expand করি:

$$\begin{aligned} & -y(1 - A_2) + A_2(1 - y) \\ & = -y + yA_2 + A_2 - A_2y \end{aligned}$$

এখানে:

$$yA_2 - A_2y = 0$$

তাই:

$$\frac{\partial L}{\partial A_2} = \frac{A_2 - y}{A_2(1 - A_2)}$$

২১.৩ দ্বিতীয় টার্ম: $\frac{\partial A_2}{\partial Z_2}$

Output layer-এ Sigmoid Activation ব্যবহার করছি:

$$A_2 = \sigma(Z_2)$$

যেখানে:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

Sigmoid-এর derivative:

$$\frac{d\sigma(z)}{dz} = \sigma(z)(1 - \sigma(z))$$

তাই:

$$\frac{\partial A_2}{\partial Z_2} = A_2(1 - A_2)$$

২১.৪ তৃতীয় টার্ম: $\frac{\partial Z_2}{\partial W_2}$ এবং $\frac{\partial Z_2}{\partial b_2}$

আমাদের weighted sum:

$$Z_2 = W_2 F + b_2$$

তাই:

$$\frac{\partial Z_2}{\partial W_2} = F$$

এবং:

$$\frac{\partial Z_2}{\partial b_2} = 1$$

২১.৫ তিনটি Term একসাথে

Chain Rule ব্যবহার করি:

$$\frac{\partial L}{\partial W_2} = \frac{\partial L}{\partial A_2} \cdot \frac{\partial A_2}{\partial Z_2} \cdot \frac{\partial Z_2}{\partial W_2}$$

উপরের তিনটি result বসালে:

$$\frac{\partial L}{\partial W_2} = \frac{A_2 - y}{A_2(1 - A_2)} \cdot A_2(1 - A_2) \cdot F$$

এখানে:

$$A_2(1 - A_2)$$

cancel হয়ে যায়।

তাই:

$$\boxed{\frac{\partial L}{\partial W_2} = (A_2 - y)F}$$

একইভাবে:

$$\begin{aligned} \frac{\partial L}{\partial b_2} &= \frac{\partial L}{\partial A_2} \cdot \frac{\partial A_2}{\partial Z_2} \cdot \frac{\partial Z_2}{\partial b_2} \\ &= \frac{A_2 - y}{A_2(1 - A_2)} \cdot A_2(1 - A_2) \cdot 1 \end{aligned}$$

তাই:

$$\boxed{\frac{\partial L}{\partial b_2} = A_2 - y}$$

২১.৬ Error Signal: δ_2

আমরা সংক্ষেপে Output Layer-এর error signal-কে বলি:

$$\boxed{\delta_2 = \frac{\partial L}{\partial Z_2}}$$

Chain Rule অনুযায়ী:

$$\frac{\partial L}{\partial Z_2} = \frac{\partial L}{\partial A_2} \cdot \frac{\partial A_2}{\partial Z_2}$$

তাই:

$$\frac{\partial L}{\partial Z_2} = \frac{A_2 - y}{A_2(1 - A_2)} \cdot A_2(1 - A_2)$$

Cancel করলে:

$$\boxed{\delta_2 = \frac{\partial L}{\partial Z_2} = A_2 - y}$$

অর্থাৎ:

$$\delta_2 = A_2 - y$$

এখানে A_2 হলো **prediction** এবং y হলো **actual target**।

শেষে আমরা পাই:

$$\frac{\partial L}{\partial W_2} = \delta_2 F$$

$$\frac{\partial L}{\partial b_2} = \delta_2$$

এটাই **Sigmoid + Binary Cross Entropy**-এর Output Layer backpropagation-এর মূল result।

অধ্যায় ২২ — Backpropagation পর্ব খ: CNN অংশ (W_1, b_1)

২২. Convolution Layer পর্যন্ত Backpropagation

এখন আমরা Output Layer থেকে পাওয়া error signal:

$$\delta_2 = \frac{\partial L}{\partial Z_2} = A_2 - y$$

পেয়ে গেছি।

এখন এই error signal-কে **উল্টো দিকে** পাঠাতে হবে:

Output → Fully Connected → Flatten → Max Pooling → ReLU → Convolution

আমাদের লক্ষ্য হলো Convolution Layer-এর weight W_1 -এর gradient বের করা।

পুরো Chain Rule:

$$\frac{\partial L}{\partial W_1} = \frac{\partial L}{\partial A_2} \frac{\partial A_2}{\partial Z_2} \frac{\partial Z_2}{\partial F} \frac{\partial F}{\partial P_1} \frac{\partial P_1}{\partial A_1} \frac{\partial A_1}{\partial Z_1} \frac{\partial Z_1}{\partial W_1}$$

প্রথম দুইটি term আমরা আগেই একত্র করে পেয়েছি:

$$\frac{\partial L}{\partial Z_2} = \delta_2 = A_2 - y$$

এখন বাকি অংশগুলো একে একে বের করব।

২২.১ ধাপ ১: Fully Connected Layer-এর Backward

আমাদের Fully Connected Layer-এর equation:

$$Z_2 = W_2 F + b_2$$

এখানে F হলো **Flatten Layer থেকে আসা feature vector**।

F -এর সাপেক্ষে Z_2 -এর derivative:

$$\frac{\partial Z_2}{\partial F} = W_2$$

তাই Chain Rule অনুযায়ী:

$$\frac{\partial L}{\partial F} = \frac{\partial L}{\partial Z_2} \frac{\partial Z_2}{\partial F}$$

অর্থাৎ:

$$\frac{\partial L}{\partial F} = \delta_2 W_2$$

এখানে δ_2 একটি scalar এবং W_2 একটি vector।

তাই:

$$\frac{\partial L}{\partial F} = (A_2 - y) W_2$$

অর্থাৎ Output Layer থেকে পাওয়া একটি error signal, W_2 -এর মাধ্যমে Flatten Layer-এর প্রতিটি feature-এর জন্য gradient তৈরি করে।

২২.২ ধাপ ২: Flatten Layer-এর Backward

Flatten Layer কোনো mathematical transformation করে না।

এটি শুধু data-এর shape পরিবর্তন করে।

Forward:

$$P_1 \rightarrow F$$

যেমন:

$$2 \times 2 \rightarrow 1 \times 4$$

ধরি:

$$P_1 = \begin{bmatrix} p_1 & p_2 \\ p_3 & p_4 \end{bmatrix}$$

Flatten করার পর:

$$F = [p_1 \quad p_2 \quad p_3 \quad p_4]$$

Backward pass-এ শুধু আগের shape-এ ফিরে যায়:

$$\frac{\partial L}{\partial F} \rightarrow \frac{\partial L}{\partial P_1}$$

অর্থাৎ:

$$\frac{\partial L}{\partial P_1} = \text{reshape} \left(\frac{\partial L}{\partial F} \right)$$

Gradient-এর মান পরিবর্তন হয় না, শুধু shape পরিবর্তন হয়।

২২.৩ ধাপ ৩: Max Pooling-এর Backward

Max Pooling-এর কোনো trainable parameter নেই।

Forward pass-এ একটি pooling window-এর মধ্যে যে value maximum ছিল, backward pass-এ শুধু সেই position-এই gradient যাবে।

ধরি একটি 2×2 window:

$$\begin{bmatrix} 3 & 1 \\ 5 & 2 \end{bmatrix}$$

Forward:

$$\max(3, 1, 5, 2) = 5$$

তাই:

$$P = 5$$

ধরি backward-এ এই output-এর gradient হলো:

$$\frac{\partial L}{\partial P} = g$$

যেহেতু 5 maximum ছিল, তাই gradient শুধু 5-এর position-এ যাবে:

$$\frac{\partial L}{\partial A} = \begin{bmatrix} 0 & 0 \\ g & 0 \end{bmatrix}$$

অর্থাৎ:

$$\frac{\partial P}{\partial A_{i,j}} = \begin{cases} 1, & \text{যদি } A_{i,j} \text{ maximum হয়} \\ 0, & \text{অন্যথায়} \end{cases}$$

তাই:

$$\frac{\partial L}{\partial A_1} = \frac{\partial L}{\partial P_1} \odot \frac{\partial P_1}{\partial A_1}$$

এখানে $\odot$ হলো **element-wise multiplication**।

কেন শুধু maximum position gradient পায়?

কারণ:

$$P = \max(a, b, c, d)$$

যদি c maximum হয়, তাহলে P মূলত c -এর ওপর নির্ভর করছে।

তাই:

$$\frac{\partial P}{\partial c} = 1$$

কিন্তু:

$$\frac{\partial P}{\partial a} = \frac{\partial P}{\partial b} = \frac{\partial P}{\partial d} = 0$$

Average Pooling হলে

Average Pooling-এর ক্ষেত্রে প্রতিটি value সমানভাবে output-এ contribute করে।

যদি:

$$P = \frac{1}{k^2} \sum A$$

তাহলে প্রতিটি input-এর derivative:

$$\frac{\partial P}{\partial A} = \frac{1}{k^2}$$

তাই backward-এ একটি gradient g প্রতিটি position-এ ভাগ হয়ে যায়:

$$\frac{g}{k^2}$$

২২.৪ ধাপ ৪: ReLU-এর Backward

Forward pass-এ:

$$A_1 = \text{ReLU}(Z_1)$$

এবং:

$$\text{ReLU}(z) = \max(0, z)$$

ReLU-এর derivative:

$$\frac{\partial A_1}{\partial Z_1} = \begin{cases} 1, & Z_1 > 0 \\ 0, & Z_1 \leq 0 \end{cases}$$

অর্থাৎ:

যদি $Z_1 > 0$

$$\frac{\partial A_1}{\partial Z_1} = 1$$

তাই gradient unchanged অবস্থায় pass করবে।

যদি $Z_1 \leq 0$

$$\frac{\partial A_1}{\partial Z_1} = 0$$

তাই gradient বন্ধ হয়ে যাবে।

সুতরাং:

$$\frac{\partial L}{\partial Z_1} = \frac{\partial L}{\partial A_1} \odot \text{ReLU}'(Z_1)$$

যেখানে:

$$\text{ReLU}'(Z_1) = \begin{cases} 1, & Z_1 > 0 \\ 0, & Z_1 \leq 0 \end{cases}$$

অর্থাৎ Forward pass-এ যেখানে ReLU positive ছিল, সেখানে backward gradient pass করবে। আর যেখানে ReLU output 0 হয়েছিল, সেখানে gradient 0 হয়ে যাবে।

অধ্যায় ২৩ — Convolution Layer-এর Gradient (সবচেয়ে গুরুত্বপূর্ণ অংশ)

এখন পর্যন্ত Backpropagation করে আমরা পেয়েছি:

$$\frac{\partial L}{\partial Z_1}$$

এখন আমাদের মূল লক্ষ্য হলো:

$$\frac{\partial L}{\partial W_1}$$

এবং

$$\frac{\partial L}{\partial b_1}$$

অর্থাৎ, Convolution Layer-এর Weight এবং Bias কতটা পরিবর্তন করলে Loss পরিবর্তিত হবে, সেটি বের করা।

২৩.১ একটি ছোট উদাহরণ

ধরি:

$$X = \begin{bmatrix} x_{11} & x_{12} & x_{13} \\ x_{21} & x_{22} & x_{23} \\ x_{31} & x_{32} & x_{33} \end{bmatrix}$$

এবং একটি 2×2 Filter:

$$W_1 = \begin{bmatrix} w_{11} & w_{12} \\ w_{21} & w_{22} \end{bmatrix}$$

Bias:

$$b_1 = b$$

তাহলে 3×3 Input এবং 2×2 Filter-এর ক্ষেত্রে Output হবে:
 2×2

কারণ:

$$\begin{aligned} n_{\text{out}} &= n - f + 1 \\ &= 3 - 2 + 1 = 2 \end{aligned}$$

Forward Pass

প্রথম output:

$$Z_{11} = x_{11}w_{11} + x_{12}w_{12} + x_{21}w_{21} + x_{22}w_{22} + b$$

দ্বিতীয় output:

$$Z_{12} = x_{12}w_{11} + x_{13}w_{12} + x_{22}w_{21} + x_{23}w_{22} + b$$

তৃতীয় output:

$$Z_{21} = x_{21}w_{11} + x_{22}w_{12} + x_{31}w_{21} + x_{32}w_{22} + b$$

চতুর্থ output:

$$Z_{22} = x_{22}w_{11} + x_{23}w_{12} + x_{32}w_{21} + x_{33}w_{22} + b$$

এখন একটি গুরুত্বপূর্ণ বিষয় লক্ষ্য করো।

$$w_{11}$$

চারটি output-এর প্রতিটিতেই ব্যবহৃত হয়েছে।

অর্থাৎ:

$$w_{11} \rightarrow Z_{11}, Z_{12}, Z_{21}, Z_{22}$$

এটাই **Parameter Sharing**।

তাই w_{11} -এর gradient শুধু একটি জায়গা থেকে আসবে না। চারটি output-এর মাধ্যমে Loss-এ তার যে প্রভাব পড়েছে, সবগুলো যোগ করতে হবে।

২৩.২ w_{11} -এর Gradient বের করা

Multivariable Chain Rule ব্যবহার করি:

$$\frac{\partial L}{\partial w_{11}} = \frac{\partial L}{\partial Z_{11}} \frac{\partial Z_{11}}{\partial w_{11}} + \frac{\partial L}{\partial Z_{12}} \frac{\partial Z_{12}}{\partial w_{11}} + \frac{\partial L}{\partial Z_{21}} \frac{\partial Z_{21}}{\partial w_{11}} + \frac{\partial L}{\partial Z_{22}} \frac{\partial Z_{22}}{\partial w_{11}}$$

Forward equations থেকে:

$$\begin{aligned} \frac{\partial Z_{11}}{\partial w_{11}} &= x_{11} \\ \frac{\partial Z_{12}}{\partial w_{11}} &= x_{12} \\ \frac{\partial Z_{21}}{\partial w_{11}} &= x_{21} \\ \frac{\partial Z_{22}}{\partial w_{11}} &= x_{22} \end{aligned}$$

তাই:

$$\frac{\partial L}{\partial w_{11}} = \frac{\partial L}{\partial Z_{11}} x_{11} + \frac{\partial L}{\partial Z_{12}} x_{12} + \frac{\partial L}{\partial Z_{21}} x_{21} + \frac{\partial L}{\partial Z_{22}} x_{22}$$

২৩.৩ সব Weight-এর Gradient

একইভাবে w_{12} -এর জন্য:

$$\frac{\partial L}{\partial w_{12}} = \frac{\partial L}{\partial Z_{11}} x_{12} + \frac{\partial L}{\partial Z_{12}} x_{13} + \frac{\partial L}{\partial Z_{21}} x_{22} + \frac{\partial L}{\partial Z_{22}} x_{23}$$

w_{21} -এর জন্য:

$$\frac{\partial L}{\partial w_{21}} = \frac{\partial L}{\partial Z_{11}} x_{21} + \frac{\partial L}{\partial Z_{12}} x_{22} + \frac{\partial L}{\partial Z_{21}} x_{31} + \frac{\partial L}{\partial Z_{22}} x_{32}$$

w_{22} -এর জন্য:

$$\frac{\partial L}{\partial w_{22}} = \frac{\partial L}{\partial Z_{11}} x_{22} + \frac{\partial L}{\partial Z_{12}} x_{23} + \frac{\partial L}{\partial Z_{21}} x_{32} + \frac{\partial L}{\partial Z_{22}} x_{33}$$

এখন এগুলো একসাথে Matrix আকারে লিখলে:

$$\frac{\partial L}{\partial W_1} = \begin{bmatrix} \frac{\partial L}{\partial w_{11}} & \frac{\partial L}{\partial w_{12}} \\ \frac{\partial L}{\partial w_{21}} & \frac{\partial L}{\partial w_{22}} \end{bmatrix}$$

২৩.৪ Gradient-কে Convolution-এর মতো দেখা

এখন:

$$\frac{\partial L}{\partial Z_1} = \begin{bmatrix} \frac{\partial L}{\partial Z_{11}} & \frac{\partial L}{\partial Z_{12}} \\ \frac{\partial L}{\partial Z_{21}} & \frac{\partial L}{\partial Z_{22}} \end{bmatrix}$$

ধরি এটিকে:

$$G_Z = \frac{\partial L}{\partial Z_1}$$

বলা হলো।

তাহলে Weight Gradient-এর প্রতিটি element মূলত Input-এর একটি region এবং G_Z -এর element-wise multiplication ও summation থেকে আসে।

তাই সংক্ষেপে:

$$\frac{\partial L}{\partial W_1} = X \star \frac{\partial L}{\partial Z_1}$$

এখানে $\star$ দিয়ে **cross-correlation operation** বোঝানো নিরাপদ।

অর্থাৎ:

$$\text{Weight Gradient} = \text{Input} \star \text{Upstream Gradient}$$

এখানে filter rotate করার প্রয়োজন নেই।

২৩.৫ Bias-এর Gradient

এখন Bias-এর gradient বের করি।

প্রতিটি output equation-এ একই bias b একবার করে যোগ হয়েছে:

$$Z_{11} = \dots + b$$

$$Z_{12} = \dots + b$$

$$Z_{21} = \dots + b$$

$$Z_{22} = \dots + b$$

তাই:

$$\frac{\partial Z_{11}}{\partial b} = 1$$

$$\frac{\partial Z_{12}}{\partial b} = 1$$

$$\frac{\partial Z_{21}}{\partial b} = 1$$

$$\frac{\partial Z_{22}}{\partial b} = 1$$

Chain Rule অনুযায়ী:

$$\frac{\partial L}{\partial b} = \frac{\partial L}{\partial Z_{11}} + \frac{\partial L}{\partial Z_{12}} + \frac{\partial L}{\partial Z_{21}} + \frac{\partial L}{\partial Z_{22}}$$

সাধারণভাবে:

$$\frac{\partial L}{\partial b_1} = \sum_i \sum_j \frac{\partial L}{\partial Z_{1,i,j}}$$

অর্থাৎ পুরো Feature Map-এর সব gradient যোগ করলেই Bias-এর gradient পাওয়া যায়।

২৩.৬ Input-এর Gradient: $\frac{\partial L}{\partial X}$

এখনও একটি বিষয় বাকি।

যদি X শুধু original image হয়, তাহলে এটি trainable parameter নয়। কিন্তু Deep CNN-এ একটি Convolution Layer-এর X সাধারণত আগের Layer-এর output।

তাই gradient আরও পেছনে পাঠাতে হলে প্রয়োজন:

$$\frac{\partial L}{\partial X}$$

এটি বের করার সময় Filter-এর spatial orientation গুরুত্বপূর্ণ হয়ে যায়।

Mathematical convolution ব্যবহার করলে:

$$\frac{\partial L}{\partial X} = \frac{\partial L}{\partial Z_1} * \text{rot180}(W_1)$$

এখানে:

$$\text{rot180}(W_1)$$

মানে W_1 -কে 180° rotate করা।

২৩.৭ তিনটি Gradient আলাদা করে মনে রাখো

Convolution Layer-এর backward pass-এ মূলত তিনটি gradient গুরুত্বপূর্ণ:

Weight-এর জন্য

$$\frac{\partial L}{\partial W_1} = X * \frac{\partial L}{\partial Z_1}$$

Bias-এর জন্য

$$\frac{\partial L}{\partial b_1} = \sum_i \sum_j \frac{\partial L}{\partial Z_{1,i,j}}$$

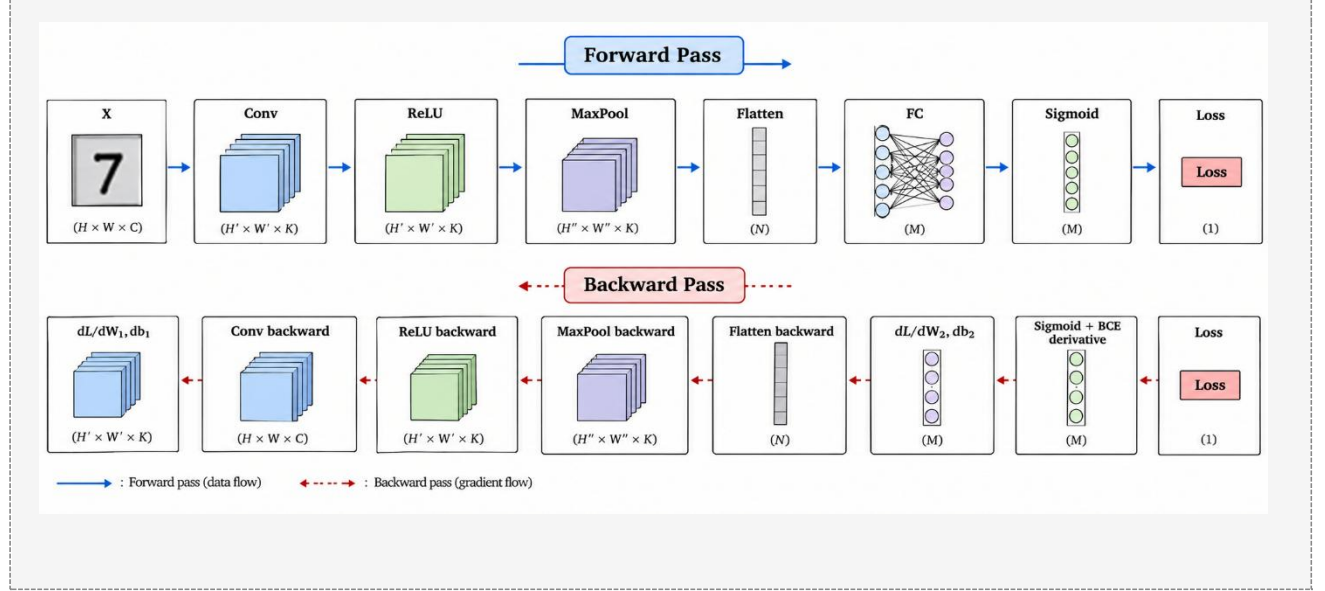
Input-এর জন্য

$$\frac{\partial L}{\partial X} = \frac{\partial L}{\partial Z_1} * \text{rot180}(W_1)$$

সবচেয়ে গুরুত্বপূর্ণ ধারণাটি হলো:

একই Weight অনেক জায়গায় ব্যবহৃত হয় $\Rightarrow$ সব জায়গার gradient যোগ করতে হয়

এটাই **Parameter Sharing**-এর কারণে **Convolutional Layer**-এর **backward propagation**-এর মূল ধারণা।



অধ্যায় ২৪ — সম্পূর্ণ Backpropagation প্রবাহ (সারসংক্ষেপ)

আমরা **Loss** L থেকে শুরু করে gradient-কে ধাপে ধাপে পেছনের দিকে পাঠাই:

$$L \rightarrow Z_2 \rightarrow F \rightarrow P_1 \rightarrow A_1 \rightarrow Z_1 \rightarrow W_1, b_1$$

ধাপ	কী বের করছি	মূল গণনা
১	Output error signal	$\delta_2 = \frac{\partial L}{\partial Z_2} = A_2 - y$
২	FC Layer-এর gradients	$\frac{\partial L}{\partial W_2} = \delta_2 F, \frac{\partial L}{\partial b_2} = \delta_2$
৩	Flatten-এর input gradient	$\frac{\partial L}{\partial F} = \delta_2 W_2$
৪	Flatten-এর backward	$\frac{\partial L}{\partial P_1} = \text{reshape} \left(\frac{\partial L}{\partial F} \right)$
৫	Max Pooling-এর backward	শুধু maximum position-এ gradient যায়
৬	ReLU-এর backward	$\frac{\partial L}{\partial Z_1} = \frac{\partial L}{\partial A_1} \odot \text{ReLU}'(Z_1)$
৭	Convolution Weight gradient	$\frac{\partial L}{\partial W_1} = X * \frac{\partial L}{\partial Z_1}$
৮	Convolution Bias gradient	$\frac{\partial L}{\partial b_1} = \sum_i \sum_j \frac{\partial L}{\partial Z_{1,i,j}}$

Max Pooling-এর ক্ষেত্রে

যদি একটি pooling window-এর maximum position $A_{i,j}$ হয়, তাহলে:

$$\frac{\partial P}{\partial A_{i,j}} = \begin{cases} 1, & A_{i,j} \text{ maximum হলে} \\ 0, & \text{অন্যথায়} \end{cases}$$

তাই gradient শুধু সেই maximum position-এ route হয়।

ReLU-এর ক্ষেত্রে

$$\text{ReLU}'(Z_1) = \begin{cases} 1, & Z_1 > 0 \\ 0, & Z_1 \leq 0 \end{cases}$$

তাই:

$$\frac{\partial L}{\partial Z_1} = \frac{\partial L}{\partial A_1} \odot \text{ReLU}'(Z_1)$$

শেষ পর্যন্ত Convolution Layer

$$\frac{\partial L}{\partial W_1} = X \star \frac{\partial L}{\partial Z_1}$$

$$\frac{\partial L}{\partial b_1} = \sum_i \sum_j \frac{\partial L}{\partial Z_{1,i,j}}$$

অর্থাৎ পুরো Backward Pass-এর সবচেয়ে সংক্ষিপ্ত মানসিক ছবি:

$$\delta_2 \rightarrow \frac{\partial L}{\partial F} \rightarrow \frac{\partial L}{\partial P_1} \rightarrow \frac{\partial L}{\partial A_1} \rightarrow \frac{\partial L}{\partial Z_1} \rightarrow \frac{\partial L}{\partial W_1}, \frac{\partial L}{\partial b_1}$$

এখানে প্রতিটি ধাপে আগের ধাপের gradient-কে সেই layer-এর local derivative/operation-এর মাধ্যমে আরও পেছনে পাঠানো হচ্ছে। এটাই Backpropagation-এর মূল ধারণা।

অধ্যায় ২৫ — সম্পূর্ণ সংখ্যাসহ Backpropagation উদাহরণ

Parameter Update

এই অধ্যায়ে আমরা একটি ছোট CNN-এর **সম্পূর্ণ Training Step** হাতে-কলমে সম্পন্ন করব।
আমরা দেখব:

$$\text{Forward Propagation} \rightarrow \text{Loss} \rightarrow \text{Backward Propagation} \rightarrow \text{Parameter Update}$$

এখানে প্রতিটি ধাপের numerical calculation এমনভাবে করা হবে যাতে কাগজে-কলমে আবার যাচাই করা যায়।

২৫.১ Setup**Input**

$$X = \begin{bmatrix} 2 & 1 & 0 \\ 1 & 3 & 2 \\ 0 & 1 & 4 \end{bmatrix}$$

Input-এর shape:

$$\boxed{3 \times 3}$$

Convolution Filter

$$W_1 = \begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix}$$

Bias:

$$\boxed{b_1 = 0.5}$$

Filter-এর shape:

$$\boxed{2 \times 2}$$

Fully Connected Layer

$$\boxed{W_2 = 2}$$

$$\boxed{b_2 = -1}$$

Target

$$\boxed{y = 1}$$

Learning Rate

$$\boxed{\eta = 0.1}$$

২৫.২ Step 1: Convolution

Output size:

$$\begin{aligned} n_{\text{out}} &= n - f + 1 \\ &= 3 - 2 + 1 \\ &= 2 \end{aligned}$$

তাই:

$$Z_1 \in \mathbb{R}^{2 \times 2}$$

প্রথম position:

$$\begin{aligned} Z_{1,1} &= 2(1) + 1(0) + 1(0) + 3(-1) + 0.5 \\ &= 2 - 3 + 0.5 \\ \boxed{Z_{1,1} &= -0.5} \end{aligned}$$

দ্বিতীয় position:

$$\begin{aligned} Z_{1,2} &= 1(1) + 0(0) + 3(0) + 2(-1) + 0.5 \\ &= 1 - 2 + 0.5 \\ \boxed{Z_{1,2} &= -0.5} \end{aligned}$$

তৃতীয় position:

$$Z_{2,1} = 1(1) + 3(0) + 0(0) + 1(-1) + 0.5$$

$$= 1 - 1 + 0.5$$

$$\boxed{Z_{2,1} = 0.5}$$

চতুর্থ position:

$$Z_{2,2} = 3(1) + 2(0) + 1(0) + 4(-1) + 0.5$$

$$= 3 - 4 + 0.5$$

$$\boxed{Z_{2,2} = -0.5}$$

সুতরাং:

$$\boxed{Z_1 = \begin{bmatrix} -0.5 & -0.5 \\ 0.5 & -0.5 \end{bmatrix}}$$

২৫.৩ Step 2: ReLU

ReLU:

$$\text{ReLU}(z) = \max(0, z)$$

তাই:

$$A_1 = \text{ReLU}(Z_1)$$

$$\boxed{A_1 = \begin{bmatrix} 0 & 0 \\ 0.5 & 0 \end{bmatrix}}$$

২৫.৪ Step 3: Max Pooling

পুরো 2×2 matrix-কে একটি pooling window হিসেবে ধরা হয়েছে।

$$P_1 = \max(0, 0, 0.5, 0)$$

তাই:

$$\boxed{P_1 = 0.5}$$

Max value এসেছে:

$$\boxed{A_1[2, 1] = 0.5}$$

এই **max location** backward pass-এর জন্য মনে রাখতে হবে।

২৫.৫ Step 4: Flatten

Pooling output একটি scalar, তাই Flatten করার পর:

$$\boxed{F = [0.5]}$$

২৫.৬ Step 5: Fully Connected Layer

$$Z_2 = W_2 F + b_2$$

মান বসালে:

$$Z_2 = 2(0.5) - 1$$

$$= 1 - 1$$

$$\boxed{Z_2 = 0}$$

২৫.৭ Step 6: Sigmoid

$$A_2 = \sigma(Z_2)$$

যেখানে:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

তাই:

$$\begin{aligned} A_2 &= \frac{1}{1 + e^0} \\ &= \frac{1}{2} \\ \boxed{A_2 = 0.5} \end{aligned}$$

অর্থাৎ model-এর prediction:

$$\boxed{50\%}$$

২৫.৮ Step 7: Loss

Binary Cross Entropy:

$$L = -y \ln(A_2) - (1 - y) \ln(1 - A_2)$$

যেহেতু:

$$y = 1$$

তাই:

$$\begin{aligned} L &= -\ln(0.5) \\ \boxed{L \approx 0.6931} \end{aligned}$$

২৫.৯ Step 8: Output Error Signal

Sigmoid + BCE-এর simplified result:

$$\boxed{\delta_2 = \frac{\partial L}{\partial Z_2} = A_2 - y}$$

তাই:

$$\begin{aligned} \delta_2 &= 0.5 - 1 \\ \boxed{\delta_2 = -0.5} \end{aligned}$$

২৫.১০ Step 9: W_2 এবং b_2 -এর Gradient

Weight gradient:

$$\begin{aligned} \frac{\partial L}{\partial W_2} &= \delta_2 F \\ &= (-0.5)(0.5) \\ \boxed{\frac{\partial L}{\partial W_2} = -0.25} \end{aligned}$$

Bias gradient:

$$\boxed{\frac{\partial L}{\partial b_2} = \delta_2 = -0.5}$$

২৫.১১ Step 10: Flatten-এর দিকে Gradient

$$\begin{aligned}\frac{\partial L}{\partial F} &= \delta_2 W_2 \\ &= (-0.5)(2) \\ \boxed{\frac{\partial L}{\partial F} &= -1}\end{aligned}$$

২৫.১২ Step 11: Flatten Backward

Flatten শুধু shape পরিবর্তন করে।

তাই gradient-এর numerical value পরিবর্তন হবে না:

$$\boxed{\frac{\partial L}{\partial P_1} = [-1]}$$

২৫.১৩ Step 12: Max Pooling Backward

Forward pass-এ maximum ছিল:

$$A_1[2, 1] = 0.5$$

তাই পুরো gradient:

$$-1$$

শুধু সেই position-এ যাবে।

সুতরাং:

$$\boxed{\frac{\partial L}{\partial A_1} = \begin{bmatrix} 0 & 0 \\ -1 & 0 \end{bmatrix}}$$

২৫.১৪ Step 13: ReLU Backward

ReLU derivative:

$$\text{ReLU}'(z) = \begin{cases} 1, & z > 0 \\ 0, & z \leq 0 \end{cases}$$

আমাদের:

$$Z_1 = \begin{bmatrix} -0.5 & -0.5 \\ 0.5 & -0.5 \end{bmatrix}$$

তাই:

$$\text{ReLU}'(Z_1) = \begin{bmatrix} 0 & 0 \\ 1 & 0 \end{bmatrix}$$

এখন:

$$\frac{\partial L}{\partial Z_1} = \frac{\partial L}{\partial A_1} \odot \text{ReLU}'(Z_1)$$

অর্থাৎ:

$$\frac{\partial L}{\partial Z_1} = \begin{bmatrix} 0 & 0 \\ -1 & 0 \end{bmatrix} \odot \begin{bmatrix} 0 & 0 \\ 1 & 0 \end{bmatrix}$$

তাই:

$$\boxed{\frac{\partial L}{\partial Z_1} = \begin{bmatrix} 0 & 0 \\ -1 & 0 \end{bmatrix}}$$

২৫.১৫ Step 14: Convolution Weight Gradient

এখন সবচেয়ে গুরুত্বপূর্ণ অংশ।

আমরা জানি:

$$\frac{\partial L}{\partial Z_1} = \begin{bmatrix} 0 & 0 \\ -1 & 0 \end{bmatrix}$$

শুধু $Z_{2,1}$ -এর gradient non-zero:

$$\frac{\partial L}{\partial Z_{2,1}} = -1$$

$Z_{2,1}$ তৈরি হয়েছিল Input-এর এই 2×2 patch থেকে:

$$\begin{bmatrix} 1 & 3 \\ 0 & 1 \end{bmatrix}$$

তাই Weight Gradient:

$$\frac{\partial L}{\partial W_1} = (-1) \begin{bmatrix} 1 & 3 \\ 0 & 1 \end{bmatrix}$$

অর্থাৎ:

$$\boxed{\frac{\partial L}{\partial W_1} = \begin{bmatrix} -1 & -3 \\ 0 & -1 \end{bmatrix}}$$

২৫.১৬ Step 15: Bias Gradient

Convolution-এর একই bias প্রতিটি output position-এ ব্যবহৃত হয়েছে।

তাই:

$$\frac{\partial L}{\partial b_1} = \sum_i \sum_j \frac{\partial L}{\partial Z_{1,i,j}}$$

অর্থাৎ:

$$\frac{\partial L}{\partial b_1} = 0 + 0 - 1 + 0$$

$$\boxed{\frac{\partial L}{\partial b_1} = -1}$$

২৫.১৭ Step 16: Parameter Update

Gradient Descent-এর নিয়ম:

$$\boxed{W_{\text{new}} = W_{\text{old}} - \eta \frac{\partial L}{\partial W}}$$

এবং:

$$\boxed{b_{\text{new}} = b_{\text{old}} - \eta \frac{\partial L}{\partial b}}$$

যেখানে:

$$\eta = 0.1$$

W_1 Update

$$W_{1,11}^{new} = 1 - 0.1(-1) = 1.1$$

$$W_{1,12}^{new} = 0 - 0.1(-3) = 0.3$$

$$W_{1,21}^{new} = 0 - 0.1(0) = 0$$

$$W_{1,22}^{new} = -1 - 0.1(-1) = -0.9$$

তাই:

$$W_1^{new} = \begin{bmatrix} 1.1 & 0.3 \\ 0 & -0.9 \end{bmatrix}$$

 b_1 Update

$$b_1^{new} = 0.5 - 0.1(-1)$$

$$b_1^{new} = 0.6$$

 W_2 Update

$$W_2^{new} = 2 - 0.1(-0.25)$$

$$W_2^{new} = 2.025$$

 b_2 Update

$$b_2^{new} = -1 - 0.1(-0.5)$$

$$b_2^{new} = -0.95$$

২৫.১৮ পুরো Training Step এক নজরে

Forward Pass:

$$X \rightarrow Z_1 \rightarrow A_1 \rightarrow P_1 \rightarrow F \rightarrow Z_2 \rightarrow A_2 \rightarrow L$$

Backward Pass:

$$L \rightarrow \delta_2 \rightarrow \frac{\partial L}{\partial W_2}, \frac{\partial L}{\partial b_2} \rightarrow \frac{\partial L}{\partial F} \rightarrow \frac{\partial L}{\partial P_1} \rightarrow \frac{\partial L}{\partial A_1} \rightarrow \frac{\partial L}{\partial Z_1} \rightarrow \frac{\partial L}{\partial W_1}, \frac{\partial L}{\partial b_1}$$

শেষে Parameter Update:

$$\text{Old Parameters} \rightarrow \text{Gradients} \rightarrow \text{New Parameters}$$

Final Updated Parameters

$$W_1 = \begin{bmatrix} 1.1 & 0.3 \\ 0 & -0.9 \end{bmatrix}$$

$$b_1 = 0.6$$

$$W_2 = 2.025$$

$$b_2 = -0.95$$

এখন **পরবর্তী iteration**-এ এই নতুন parameters দিয়েই আবার Forward Propagation শুরু হবে।
অর্থাৎ পুরো চক্রটি হবে:

Forward → Loss → Backward → Update → Forward → ...

এই repeated process-এর মাধ্যমেই CNN ধীরে ধীরে এমন parameters শিখতে থাকে যেগুলো Training Data-এর Loss কমায়।